

5.栈和队列算法

队列和栈都是「操作受限」的数据结构：队列只能在一端插入元素，另一端删除元素；栈只能在某一端插入和删除元素。说白了就是把数组链表提供的 API 删掉了一部分，只保留头尾操作元素的 API 给你用。

栈和队列的实现

用链表实现栈和队列

```
// 用链表作为底层数据结构实现栈
#include <list>
#include <iostream>

template<typename E>
class MyLinkedStack {
private:
    std::list<E> list;

public:
    // 向栈顶加入元素，时间复杂度 O(1)
    void push(const E &e) {
        list.push_back(e);
    }

    // 从栈顶弹出元素，时间复杂度 O(1)
    E pop() {
        E value = list.back();
        list.pop_back();
        return value;
    }

    // 查看栈顶元素，时间复杂度 O(1)
    E peek() const {
        return list.back();
    }

    // 返回栈中的元素个数，时间复杂度 O(1)
    int size() const {
        return list.size();
    }
};
```

上面这段代码相当于是把双链表的尾部作为栈顶，在双链表尾部增删元素的时间复杂度都是 $O(1)$ ，符合要求。

```
// 用链表作为底层数据结构实现队列
```

```

template<typename E>
class MyLinkedList {
private:
    std::list<E> list;

public:
    // 向队尾插入元素，时间复杂度 O(1)
    void push(const E &e) {
        list.push_back(e);
    }

    // 从队头删除元素，时间复杂度 O(1)
    E pop() {
        E front = list.front();
        list.pop_front();
        return front;
    }

    // 查看队头元素，时间复杂度 O(1)
    E peek() {
        return list.front();
    }

    // 返回队列中的元素个数，时间复杂度 O(1)
    int size() {
        return list.size();
    }
};

```

上面这段代码相当于是把双链表的尾部作为队尾，把双链表的头部作为队头，在双链表的头尾增删元素的复杂度都是 $O(1)$ ，符合队列 API 的要求。

用数组实现栈和队列

```

// 用数组作为底层数据结构实现栈
#include <vector>

template<typename E>
class MyArrayStack {
private:
    std::vector<E> arr;

public:
    // 向栈顶加入元素，时间复杂度 O(1)
    void push(const E& e) {
        arr.push_back(e);
    }

    // 从栈顶弹出元素，时间复杂度 O(1)

```

```

    E pop() {
        E topElement = arr.back();
        arr.pop_back();
        return topElement;
    }

    // 查看栈顶元素，时间复杂度 O(1)
    E peek() const {
        return arr.back();
    }

    // 返回栈中的元素个数，时间复杂度 O(1)
    int size() const {
        return arr.size();
    }
};

```

能否让数组的头部作为栈顶？

按照我们之前实现 `MyArrayList` 的逻辑，是不行的。因为数组头部增删元素的时间复杂度都是 $O(n)$ ，不符合要求。

但是我们可以改用前文 [环形数组技巧](#) 中实现的 `CycleArray` 类，这个数据结构在头部增删元素的时间复杂度是 $O(1)$ ，符合栈的要求。

有了前文 [环形数组](#) 中实现的 `CycleArray` 类，用数组作为底层数据结构实现队列就不难了吧。直接复用我们实现的 `CycleArray`，就可以实现标准队列了。当然，一些编程语言也有内置的环形数组实现，你也可以自行搜索使用：

```

#include <iostream>
#include <deque>

template <typename E>
class MyArrayQueue {
private:
    CycleArray<E> arr;

public:
    MyArrayQueue() {
        arr = CycleArray<E>();
    }

    void push(E t) {
        arr.addLast(t);
    }

    E pop() {
        return arr.removeFirst();
    }
}

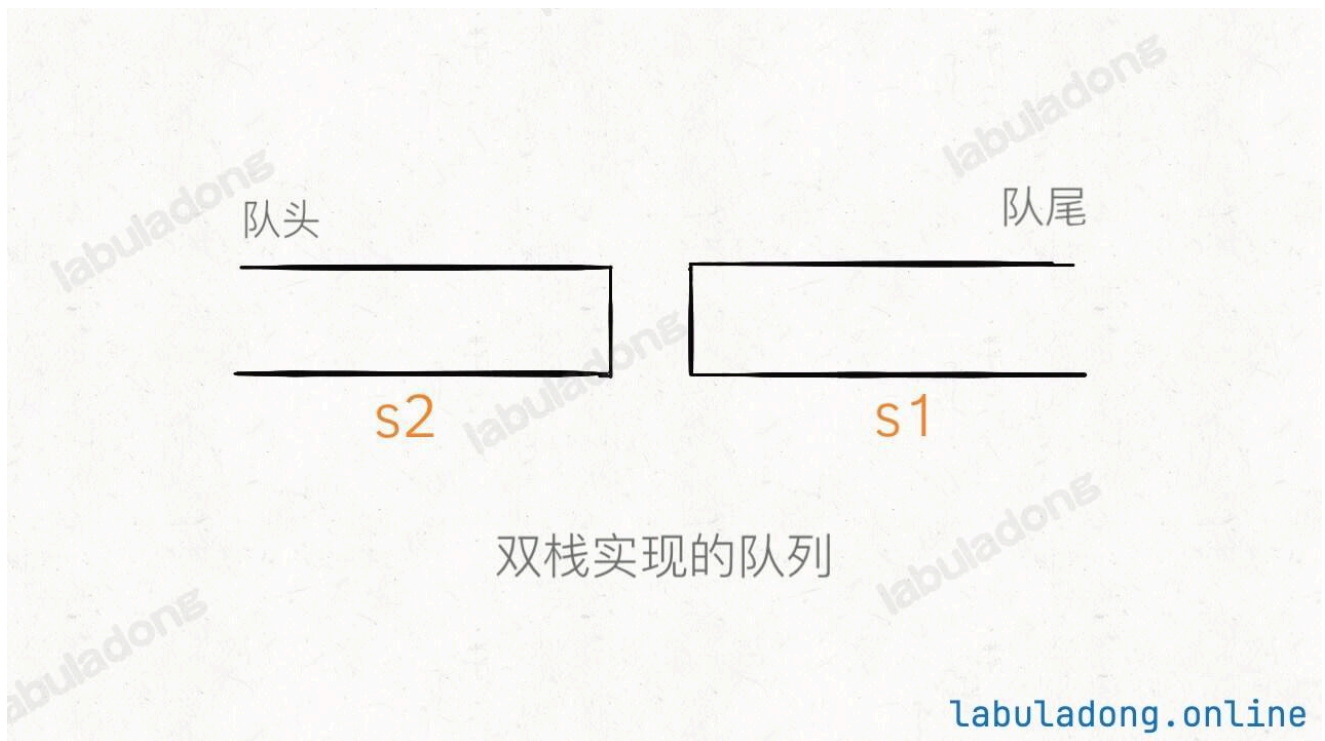
```

```
E peek() {  
    return arr.getFirst();  
}  
  
int size() {  
    return arr.size();  
}  
};
```

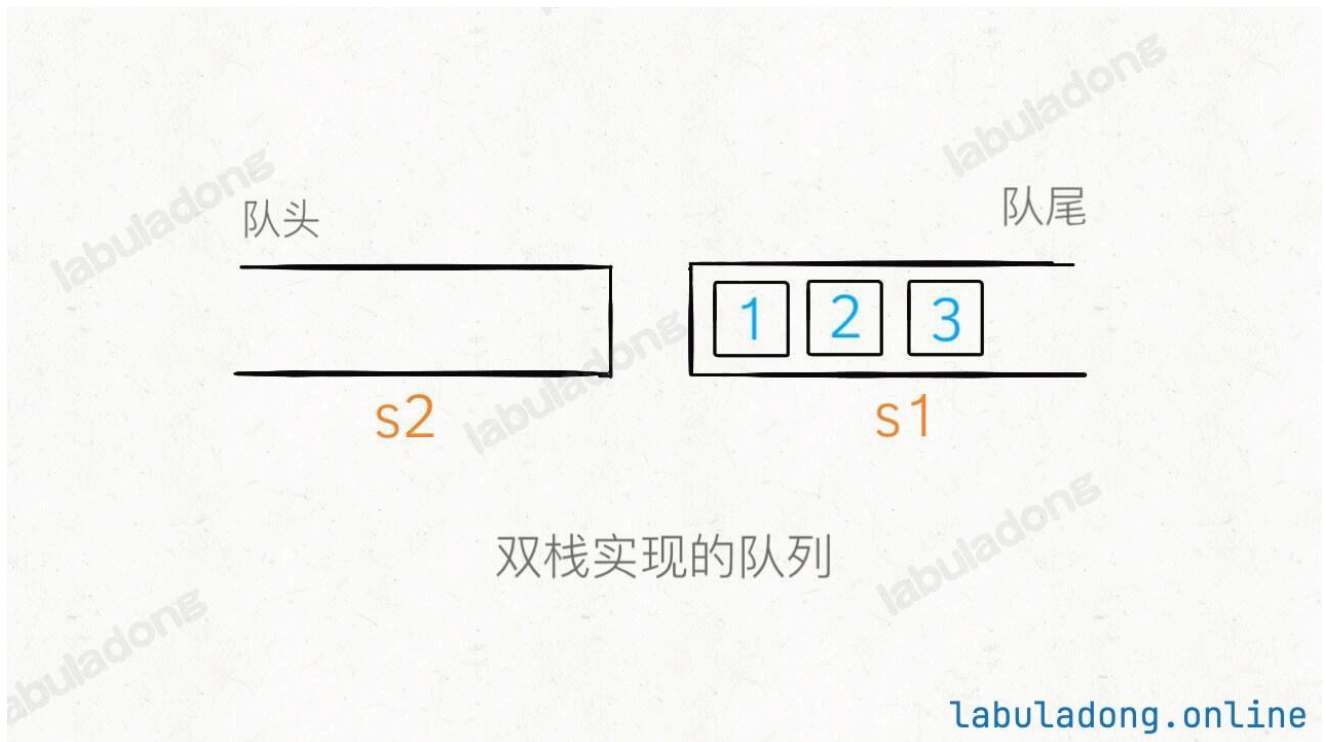
。

用栈实现队列

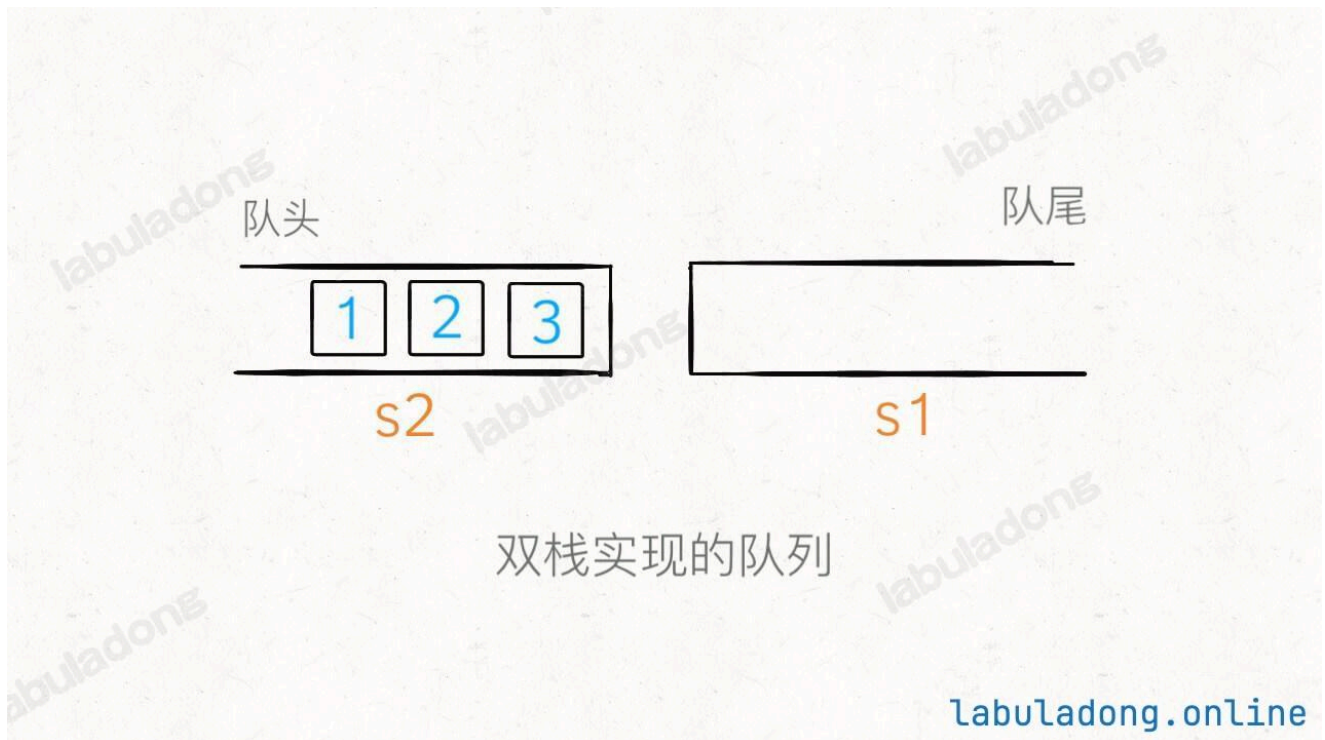
我们使用两个栈 `s1`, `s2` 就能实现一个队列的功能（这样放置栈可能更容易理解）：



当调用 `push` 让元素入队时，只要把元素压入 `s1` 即可，比如说 `push` 进 3 个元素分别是 1,2,3，那么底层结构就是这样：



那么如果这时候使用 `peek` 查看队头的元素怎么办呢？按道理队头元素应该是 1，但是在 `s1` 中 1 被压在栈底，现在就要轮到 `s2` 起到一个中转的作用了：当 `s2` 为空时，可以把 `s1` 的所有元素取出再添加进 `s2`，这时候 `s2` 中元素就是先进先出顺序了：



当 `s2` 中存在元素时，直接调用操作 `s2` 的 `pop` 方法，弹出的就是最先插入的元素，即实现了队列的 `pop` 操作。

```
class MyQueue {
private:
    stack<int> s1, s2;

public:
```

```

MyQueue() {
    // Constructor initializes two stacks
}

// 添加元素到队尾
void push(int x) {
    s1.push(x);
}

// 删除队头元素并返回
int pop() {
    // 先调用 peek 保证 s2 非空
    peek();
    int topElement = s2.top();
    s2.pop();
    return topElement;
}

// 返回队头元素
int peek() {
    if (s2.empty()) {
        // 把 s1 元素压入 s2
        while (!s1.empty()) {
            s2.push(s1.top());
            s1.pop();
        }
    }
    return s2.top();
}

// 判断队列是否为空
bool empty() {
    return s1.empty() && s2.empty();
}
};

```

至此，就用栈结构实现了一个队列，核心思想是利用两个栈互相配合。

值得一提的是，这几个操作的时间复杂度是多少呢？

有点意思的是 `peek` 操作，调用它时可能触发 `while` 循环，这样的话时间复杂度是 $O(N)$ ，但是大部分情况下 `while` 循环不会被触发，时间复杂度是 $O(1)$ 。由于 `pop` 操作调用了 `peek`，它的时间复杂度和 `peek` 相同。

像这种情况，可以说它们的最坏时间复杂度是 $O(N)$ ，因为包含 `while` 循环，可能需要从 `s1` 往 `s2` 搬移元素。

但是它们的均摊时间复杂度是 $O(1)$ ，这个要这么理解：对于一个元素，最多只可能被搬运一次，也就是说 `peek` 操作平均到每个元素的时间复杂度是 $O(1)$ 。

关于时间复杂度的分析方法，详见 [时空复杂度实用分析方法](#)。

用队列实现栈

如果说双栈实现队列比较巧妙，那么用队列实现栈就比较简单粗暴了，只需要一个队列作为底层数据结构就能实现了。

```
#include<queue>
using namespace std;

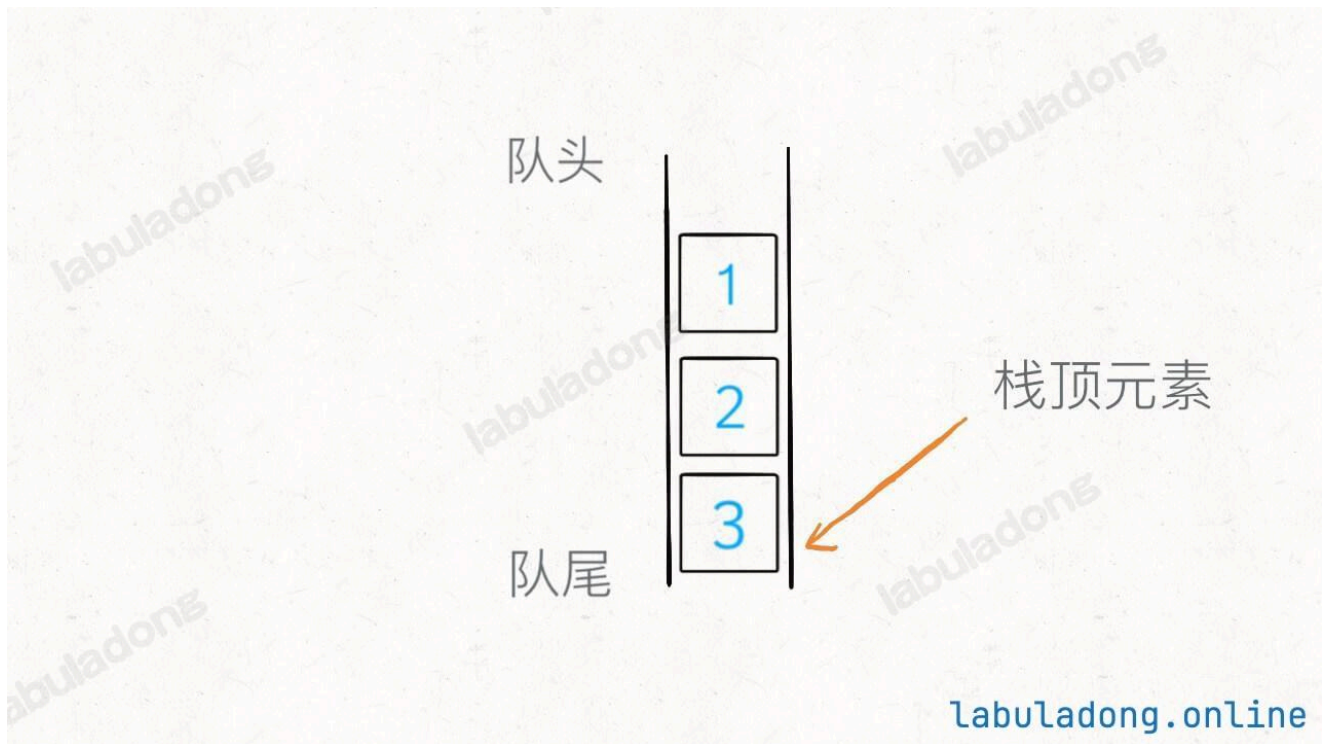
class MyStack {
    queue<int> q;
    int top_elem = 0;

public:
    // 添加元素到栈顶
    void push(int x) {
        // x 是队列的队尾，是栈的栈顶
        q.push(x);
        top_elem = x;
    }

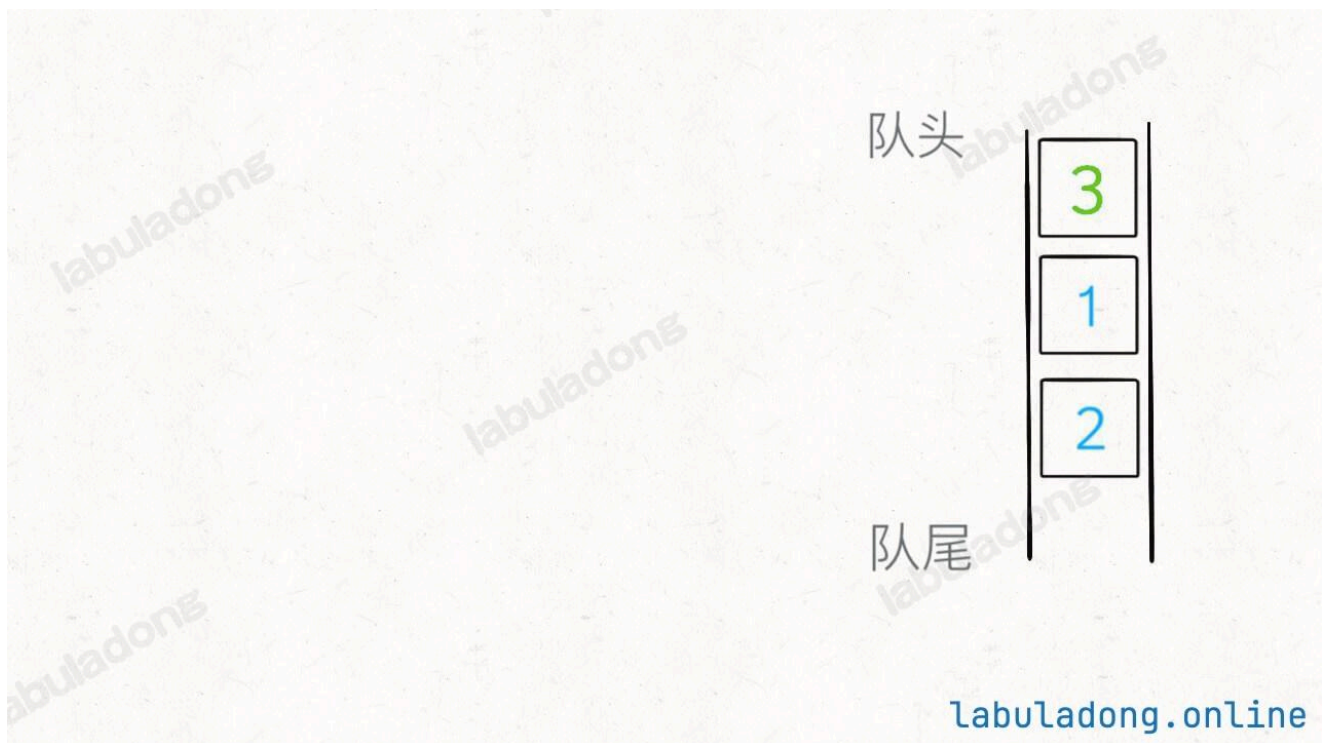
    // 返回栈顶元素
    int top() {
        return top_elem;
    }

    bool empty() {
        return q.empty();
    }
};
```

我们的底层数据结构是先进先出的队列，每次 `pop` 只能从队头取元素；但是栈是后进先出，也就是说 `pop` API 要从队尾取元素：



解决方法简单粗暴，把队列前面的都取出来再加入队尾，让之前的队尾元素排到队头，这样就可以取出了：



```
class MyStack {
    // 为了节约篇幅，省略上文给出的代码部分...

    // 删除栈顶的元素并返回
    int pop() {
        int size = q.size();
        while (size > 1) {
            q.push(q.front());
            q.pop();
        }
    }
}
```



```

        size--;
    }
    // 之前的队尾元素已经到了队头
    int res = q.front();
    q.pop();
    return res;
}
};

```

这样实现还有一点小问题就是，原来的队尾元素被推到队头并删除了，但是 `top_elem` 变量没有更新，我们还需要一点小修改：

```

class MyStack {
    // 为了节约篇幅，省略上文给出的代码部分...

    // 删除栈顶的元素并返回
public:
    int pop() {
        int size = q.size();
        // 留下队尾 2 个元素
        while (size > 2) {
            q.push(q.front());
            q.pop();
            size--;
        }
        // 记录新的队尾元素
        top_elem = q.front();
        q.push(q.front());
        q.pop();
        // 删除之前的队尾元素
        int target = q.front();
        q.pop();
        return target;
    }
};

```

这样就实现完了，完整的代码如下：

```

class MyStack {
    queue<int> q;
    int top_elem = 0;

public:
    // 将元素 x 压入栈顶
    void push(int x) {
        // x 是队列的队尾，是栈的栈顶
        q.push(x);
        top_elem = x;
    }
};

```

```

}

// 返回栈顶元素
int top() {
    return top_elem;
}

// 删除栈顶的元素并返回
int pop() {
    int size = q.size();
    // 留下队尾 2 个元素
    while (size > 2) {
        q.push(q.front());
        q.pop();
        size--;
    }
    // 记录新的队尾元素
    top_elem = q.front();
    q.push(q.front());
    q.pop();
    // 删除之前的队尾元素
    int result = q.front();
    q.pop();
    return result;
}

// 判断栈是否为空
bool empty() {
    return q.empty();
}
};

```

很明显，用队列实现栈的话，`pop` 操作时间复杂度是 $O(N)$ ，其他操作都是 $O(1)$ 。

个人认为，用队列实现栈是没啥亮点的问题，但是用双栈实现队列是值得学习的。

栈的经典习题

考察先进后出性质

1. 简化路径 | [力扣](#) | [LeetCode](#) |

给你一个字符串 `path`，表示指向某一文件或目录的 Unix 风格 **绝对路径**（以 `'/'` 开头），请你将其转化为更加简洁的规范路径。

在 Unix 风格的文件系统中，一个点（`.`）表示当前目录本身；此外，两个点（`..`）表示将目录切换到上一级（指向父目录）；两者都可以是复杂相对路径的组成部分。任意多个连续的斜杠（即，`'//'`）都被视为单个斜杠 `'/'`。对于此问题，任何其他格式的点（例如，`'...'`）均被视为文件/目录名称。

请注意，返回的 **规范路径** 必须遵循下述格式：

- 始终以斜杠 `'/'` 开头。
- 两个目录名之间必须只有一个斜杠 `'/'` 。
- 最后一个目录名（如果存在）**不能** 以 `'/'` 结尾。
- 此外，路径仅包含从根目录到目标文件或目录的路径上的目录（即，不含 `'.'` 或 `'..'`）。

返回简化后得到的 **规范路径** 。

示例 1：

输入：path = `"/home/"`

输出：`"/home"`

解释： 注意，最后一个目录名后面没有斜杠。

示例 2：

输入：path = `"/../"`

输出：`"/"`

解释： 从根目录向上一级是不可行的，因为根目录是你到达的最高级。

示例 3：

输入：path = `"/home//foo/"`

输出：`"/home/foo"`

解释： 在规范路径中，多个连续斜杠需要用一个斜杠替换。

示例 4：

输入：path = `"/a/./b/../../c/"`

输出：`"/c"`

提示：

- `1 <= path.length <= 3000`
- path 由英文字母，数字，`'.'`，`'/'` 或 `'_'` 组成。
- path 是一个有效的 Unix 风格绝对路径。

这题比较简单，利用栈先进后出的特性处理上级目录 `..`，最后组装化简后的路径即可。

```
#include <string>
#include <vector>
#include <sstream>

class Solution {
public:
    std::string simplifyPath(std::string path) {
```

```

std::vector<std::string> parts;
std::istringstream ss(path);
std::string part;

// 借助栈计算最终的文件夹路径
while (std::getline(ss, part, '/')) {
    if (part.empty() || part == ".") {
        // Skip empty parts and current directory symbol.
        continue;
    }
    if (part == "..") {
        // Go up one directory (pop from the stack) unless the stack
is empty.
        if (!parts.empty()) parts.pop_back();
    } else {
        // Add the non-empty and non-".." part to the stack.
        parts.push_back(part);
    }
}

// 栈中存储的文件夹组成路径
std::string res;
for (const auto& p : parts) {
    res += "/" + p;
}

// If the result is empty, it means the path is root directory.
return res.empty() ? "/" : res;
}
};

```

为什么用 `std::vector` 模拟栈？

在 C++ 中，`std::vector` 相比 `std::stack` 有一个重要的优势：它允许在操作完成后**方便**地遍历栈中的所有元素。

- **栈 (`std::stack`) 的局限性：** 遵循严格的 LIFO（后进先出）原则，只能通过 `push()`、`pop()` 和 `top()` 访问。它不允许您迭代内部元素。
- **Vector 模拟栈的优势：** 在这个题目中，我们处理完所有的目录和 `..` 操作后，需要将栈中剩余的目录名（例如 `["home", "foo"]`）**从头到尾**连接起来（`/home/foo`）。`std::vector` 可以通过 `push_back()` 实现入栈，通过 `pop_back()` 实现出栈，并在最后使用 `for` 循环进行遍历，非常方便地构造最终结果字符串。

143. 重排链表 | 力扣 | [LeetCode](#) |

给定一个单链表 `L` 的头节点 `head`，单链表 `L` 表示为：

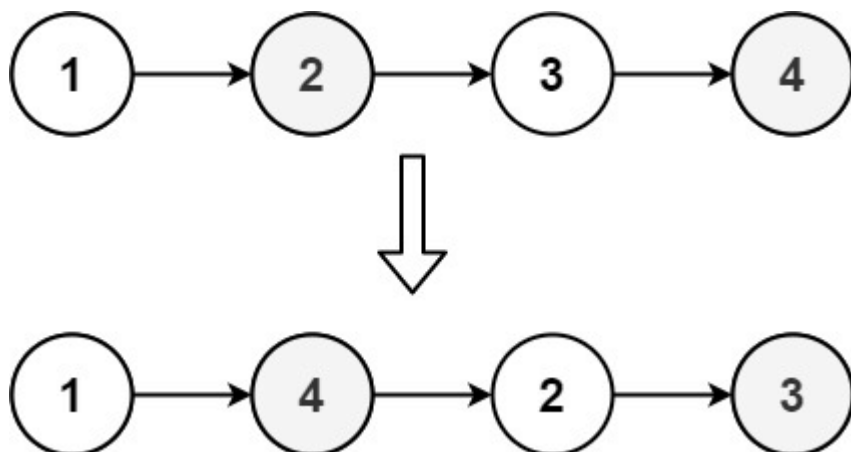
$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

请将其重新排列后变为：

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

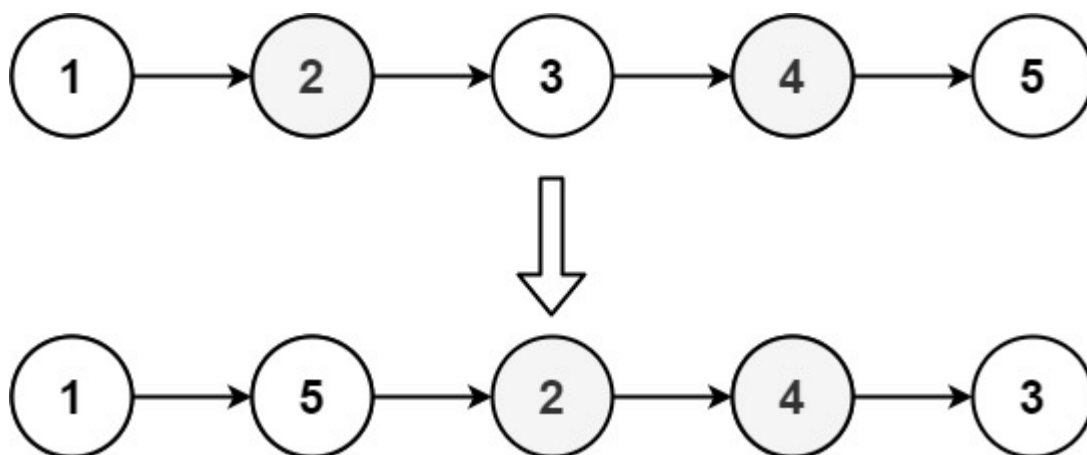
示例 1：



输入：head = [1,2,3,4]

输出：[1,4,2,3]

示例 2：



输入：head = [1,2,3,4,5]

输出：[1,5,2,4,3]

提示：

- 链表的长度范围为 $[1, 5 * 10^4]$
- $1 \leq \text{node.val} \leq 1000$

这题的难点在于：一个单链表只能从头部向尾部遍历节点，无法从尾部开始向头部遍历节点。

那么我们可以利用「栈」先进后出的结构特点，按从头到尾的顺序让链表节点入栈，那么出栈顺序就是反过来从尾到头了。

有了这个栈，算法的大致逻辑如下：

```

ListNode p = head;
while (p != null) {
    // 链表尾部的节点
    ListNode lastNode = stk.pop();
    // 按题目要求拼接
    ListNode next = p.next;
    p.next = lastNode;
    lastNode.next = next;
    p = next;
}

```

当然，处理单链表时细节问题比较多，注意操作指针时的顺序，避免操作失误形成环形链表，直接看我的代码注释吧。

```

#include <stack>

class Solution {
public:
    void reorderList(ListNode* head) {
        std::stack<ListNode*> stk;
        // 先把所有节点装进栈里，得到倒序结果
        ListNode* p = head;
        while (p != nullptr) {
            stk.push(p);
            p = p->next;
        }

        p = head;
        while (!stk.empty()) {
            // 链表尾部的节点
            ListNode* lastNode = stk.top();
            stk.pop();
            ListNode* next = p->next;
            if (lastNode == next || lastNode->next == next) {
                // 结束条件，链表节点数为奇数或偶数时均适用
                lastNode->next = nullptr;
                break;
            }
            p->next = lastNode;
            lastNode->next = next;
            p = next;
        }
    }
};

```

150. 逆波兰表达式求值 | [力扣](#) | [LeetCode](#) |

给你一个字符串数组 `tokens`，表示一个根据 [逆波兰表示法](#) 表示的算术表达式。

请你计算该表达式。返回一个表示表达式值的整数。

注意：

- 有效的算符为 '+'、'-'、'*' 和 '/' 。
- 每个操作数（运算对象）都可以是一个整数或者另一个表达式。
- 两个整数之间的除法总是 **向零截断** 。
- 表达式中不含除零运算。
- 输入是一个根据逆波兰表示法表示的算术表达式。
- 答案及所有中间计算结果可以用 **32 位** 整数表示。

示例 1：

输入：tokens = ["2","1","+","3","*"]

输出：9

解释：该算式转化为常见的中缀算术表达式为：((2 + 1) * 3) = 9

示例 2：

输入：tokens = ["4","13","5","/","+"]

输出：6

解释：该算式转化为常见的中缀算术表达式为：(4 + (13 / 5)) = 6

示例 3：

输入：tokens = ["10","6","9","3","+","-11","/","17","+","5","+"]

输出：22

解释：该算式转化为常见的中缀算术表达式为：

$$\begin{aligned} & ((10 (6 / ((9 + 3) - 11))) + 17) + 5 \\ & = ((10 (6 / (12 - 11))) + 17) + 5 \\ & = ((10 (6 / -132)) + 17) + 5 \\ & = ((10 0) + 17) + 5 \\ & = (0 + 17) + 5 \\ & = 17 + 5 \\ & = 22 \end{aligned}$$

提示：

- `1 <= tokens.length <= 104`
- `tokens[i]` 是一个算符（"+"、"-"、"*" 或 "/"），或是在范围 `[-200, 200]` 内的一个整数

逆波兰表达式：

逆波兰表达式是一种后缀表达式，所谓后缀就是指算符写在后面。

- 平常使用的算式则是一种中缀表达式，如 `( 1 + 2 ) * ( 3 + 4 )` 。

- 该算式的逆波兰表达式写法为 `(( 1 2 + ) ( 3 4 + ) * )`。

逆波兰表达式主要有以下两个优点：

- 去掉括号后表达式无歧义，上式即便写成 `1 2 + 3 4 + *` 也可以依据次序计算出正确结果。
- 适合用栈操作运算：遇到数字则入栈；遇到算符则取出栈顶两个数字进行计算，并将结果压入栈中

逆波兰表达式发明出来就是为了方便计算机运用「栈」进行表达式运算的，其运算规则如下：

按顺序遍历逆波兰表达式中的字符，如果是数字，则放入栈；如果是运算符，则将栈顶的两个元素拿出来进行运算，再将结果放入栈。对于减法和除法，运算顺序别搞反了，栈顶第二个数是被除（减）数。

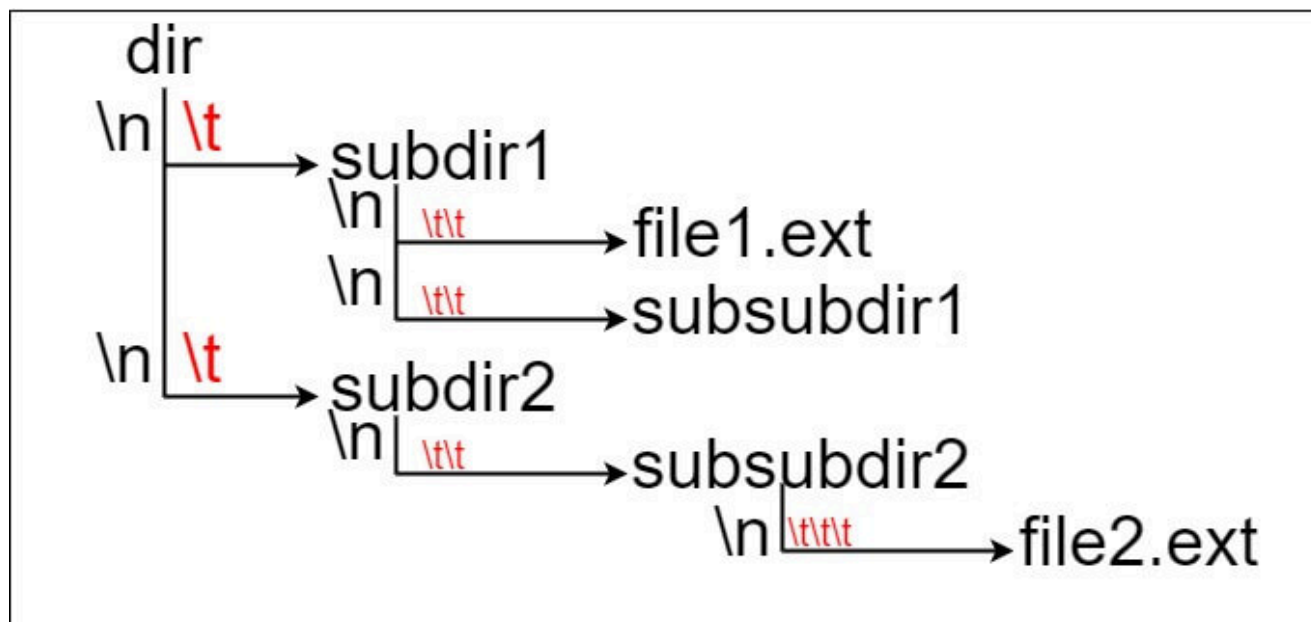
所以这题很简单，直接按照运算规则借助栈计算表达式结果即可。

```
#include <vector>
#include <string>
#include <stack>

class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        std::stack<int> stk;
        for (const std::string& token : tokens) {
            if (token == "+" || token == "-" || token == "*" || token ==
"/") {
                // 是个运算符，从栈顶拿出两个数字进行运算，运算结果入栈
                int a = stk.top(); stk.pop();
                int b = stk.top(); stk.pop();
                if (token == "+") stk.push(b + a);
                else if (token == "*") stk.push(b * a);
                else if (token == "-") {
                    // 对于减法和除法，顺序别搞反了，第二个数是被除（减）数
                    stk.push(b - a);
                }
                else if (token == "/") stk.push(b / a);
            } else {
                // 是个数字，直接入栈即可
                stk.push(std::stoi(token));
            }
        }
        // 最后栈中剩下一个数字，即是计算结果
        return stk.top();
    }
};
```

388. 文件的最长绝对路径 | 力扣 | LeetCode | ●

假设有一个同时存储文件和目录的文件系统。下图展示了文件系统的一个示例：



这里将 `dir` 作为根目录中的唯一目录。`dir` 包含两个子目录 `subdir1` 和 `subdir2`。
`subdir1` 包含文件 `file1.ext` 和子目录 `subsubdir1`；`subdir2` 包含子目录 `subsubdir2`，该子目录下包含文件 `file2.ext`。

在文本格式中，如下所示(→表示制表符)：

```
dir
→ subdir1
→ → file1.ext
→ → subsubdir1
→ subdir2
→ → subsubdir2
→ → → file2.ext
```

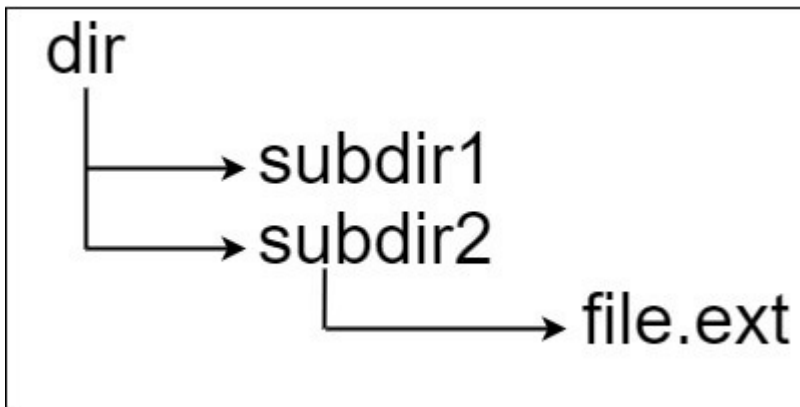
如果是代码表示，上面的文件系统可以写

为 `"dir\n\tsubdir1\n\t\tfile1.ext\n\t\tsubsubdir1\n\tsubdir2\n\t\tsubsubdir2\n\t\t\tfile2.ext"`。 `'\n'` 和 `'\t'` 分别是换行符和制表符。

文件系统在每个文件和文件夹都有一个唯一的 **绝对路径**，即必须打开才能到达文件/目录所在位置的目录顺序，所有路径用 `'/'` 连接。上面例子中，指向 `file2.ext` 的 **绝对路径** 是 `"dir/subdir2/subsubdir2/file2.ext"`。每个目录名由字母、数字和/或空格组成，每个文件名遵循 `name.extension` 的格式，其中 `name` 和 `extension` 由字母、数字和/或空格组成。

给定一个以上述格式表示文件系统的字符串 `input`，返回文件系统中 **指向文件的 最长绝对路径** 的长度。如果系统中没有文件，返回 `0`。

示例 1：

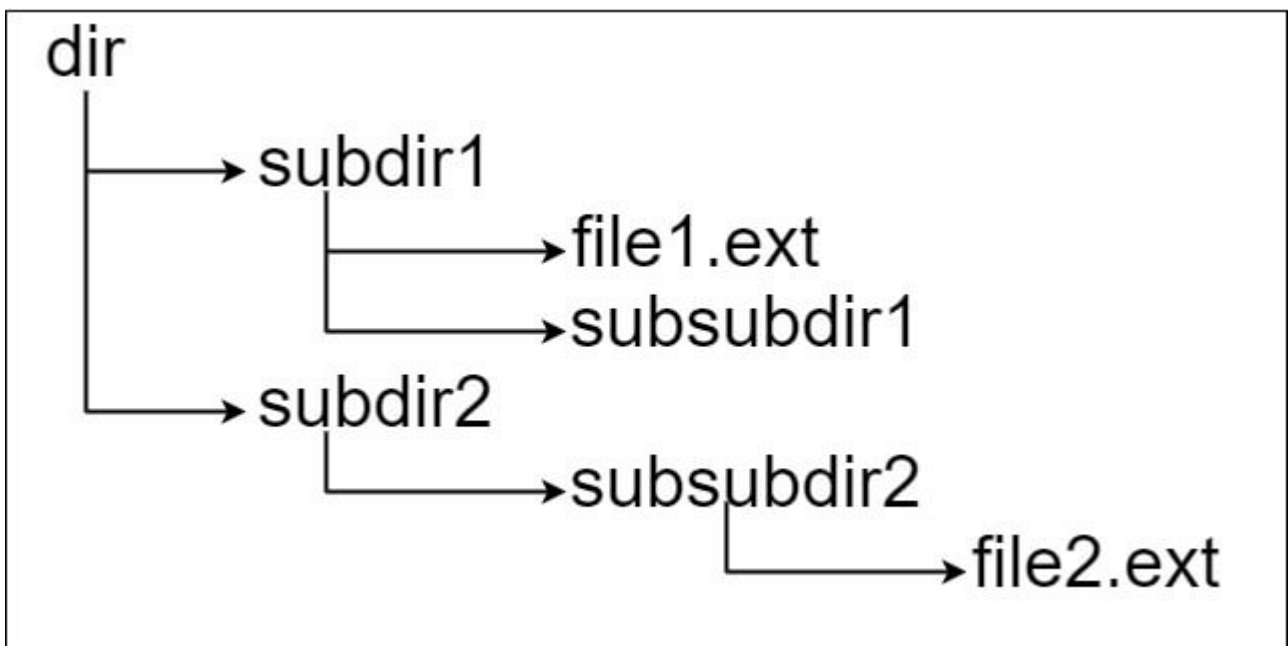


输入： input = "dir\n\tsubdir1\n\tsubdir2\n\t\tfile.ext"

输出： 20

解释： 只有一个文件，绝对路径为 "dir/subdir2/file.ext" ，路径长度 20

示例 2：



输入： input =

"dir\n\tsubdir1\n\t\tfile1.ext\n\t\t\tsubsubdir1\n\t\t\t\tsubsubdir2\n\t\t\t\t\tfile2.ext"

输出： 32

解释： 存在两个文件：

"dir/subdir1/file1.ext" ，路径长度 21

"dir/subdir2/subsubdir2/file2.ext" ，路径长度 32

返回 32 ，因为这是最长的路径

示例 3：

输入： input = "a"

输出： 0

解释： 不存在任何文件

示例 4：

输入： input = "file1.txt\nfile2.txt\nlongfile.txt"

输出： 12

解释： 根目录下有 3 个文件。

因为根目录中任何东西的绝对路径只是名称本身，所以答案是 "longfile.txt"，路径长度为 12

提示：

- `1 <= input.length <= 104`
- `input` 可能包含小写或大写的英文字母，一个换行符 `'\n'`，一个制表符 `'\t'`，一个点 `'.'`，一个空格 `' '`，和数字。

我觉得这道题还是比较实用的，因为在我做这道题之前，我就思考并解决过这个问题，可以在这里和大家分享下我的使用场景：

你可以看我的 [GitHub 仓库中的文章目录](#)，是通过缩进来表示层级的，很类似本题所说的场景。然而我需要把这些目录转化成 HTML 文档，按照文件目录的形式把这些 HTML 部署到 [我的网站](#) 上。你看，这是不是就涉及到本题生成文件的绝对路径的问题？

对于这个场景，我当时其实尝试很多可行的办法。但这里我还是写一个最简单直接容易理解的解法吧，那就是用栈来辅助，对于每一个路径，都去维护正确的父路径，从而计算最长路径的长度。具体看代码注释吧。

```
#include <deque>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    int lengthLongestPath(string input) {
        // 这个栈存储之前的父路径。实际上这里只用存父路径的长度就够了，这个优化留给你吧
        deque<int> stack;
        int maxLen = 0;
        vector<string> parts = split(input, '\n');
        for (const string& part : parts) {
            int level = part.rfind("\t") + 1;
            // '\rfind' 从字符串的末尾开始向前搜索指定子字符串或字符的最后一次出现的位置。

            // "\t\tfile.ext"最后一个 '\t' 的索引是 1，所以层级为 2
            // 降级/切换分支，让栈中只保留当前目录的父路径
            while (level < stack.size()) {
                stack.pop_back();
            }
            stack.push_back(part.size() - level);
            // 如果是文件，就计算路径长度
            if (part.find('.') != string::npos) {
                int sum = 0;
```

```

        for (int len : stack) {
            sum += len;
        }
        // 加上父路径的分隔符
        sum += stack.size() - 1;
        maxLen = max(maxLen, sum);
    }
}
return maxLen;
}

private:
    vector<string> split(const string& s, char delimiter) {
        vector<string> tokens;
        string token;
        istringstream tokenStream(s);
        while (getline(tokenStream, token, delimiter)) {
            tokens.push_back(token);
        }
        return tokens;
    }
};

```

栈的设计题

155. 最小栈 | [力扣](#) | [LeetCode](#) |

设计一个支持 `push` ， `pop` ， `top` 操作，并能在常数时间内检索到最小元素的栈。

实现 `MinStack` 类：

- `MinStack()` 初始化堆栈对象。
- `void push(int val)` 将元素`val`推入堆栈。
- `void pop()` 删除堆栈顶部的元素。
- `int top()` 获取堆栈顶部的元素。
- `int getMin()` 获取堆栈中的最小元素。

示例 1：

输入：

```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[,-2],[0],[-3],[,],[],[],[]
```

输出：

```
[null,null,null,null,-3,null,0,-2]
```

解释：

```
MinStack minStack = new MinStack();
```

```
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); --> 返回 -3.
minStack.pop();
minStack.top(); --> 返回 0.
minStack.getMin(); --> 返回 -2.
```

提示：

- `-231 <= val <= 231 - 1`
- `pop`、`top` 和 `getMin` 操作总是在 **非空栈** 上调用
- `push`、`pop`、`top`、and `getMin` 最多被调用 $3 * 10^4$ 次

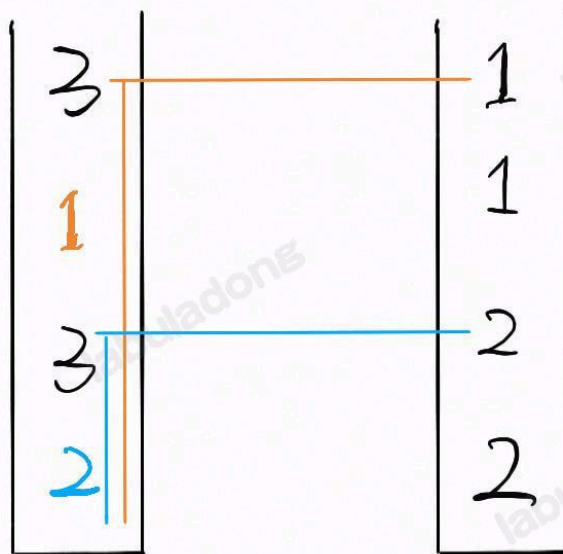
根据我们之前亲自动手实现的栈，我们知道栈是一种操作受限的数据结构，只能从栈顶插入或弹出元素，所以对于标准的栈来说，如果想实现本题的 `getMin` 方法，只能老老实实把所有元素弹出来然后找最小值。**想提高时间效率，那肯定要通过空间换时间的思路。**

不过在具体说解法之前，我想聊一下动态集合中维护最值的问题。这类问题看似简单，但实际上是个很棘手的问题。其实本题就是如下一个场景：

假设你有若干数字，你用一个 `min` 变量维护了其中的最小值，如果现在给这些数字中添加一个新数字，那么只要比较这个新数字和 `min` 的大小就可以得出最新的最小值。但如果现在从这些数字中删除一个数字，你还能用 `min` 变量得到最小值吗？答案是不能，因为如果这个被删除的数字恰好是最小值，那么新的 `min` 变量应该更新为第二小的元素对吧，但是我没有记录第二小的元素是多少，所以只能把所有数字重新遍历一遍。

明确了难点再回到本题，就可以对症下药了。删除栈顶元素的时候，不确定新的最小值是多少，但楼下那哥们知道啊，他当时入栈时的最小值，就是现在的最小值呗。

所以这道题的关键就是，**每个元素入栈时，还要记下来当前栈中的最小值**。比方说，可以用一个额外的栈 `minStk` 来记录栈中每个元素入栈时的栈中的最小元素是多少，这样每次删除元素时就能快速得到剩余栈中的最小元素了。



labuladong.online

当然，我们还可以做一些优化，减少 `minStk` 中存储的元素个数，我把原始解法和优化解法都写出来了，供参考。

PS：这道题并不难，但我还是很细致地分析了，希望你深刻理解其中的难点。下一步可以做一下 [239. 滑动窗口最大值](#)，请仔细观察和思考，队列结构是如何解决这个难点的。

```
// 原始思路
class MinStack1 {
    // 记录栈中的所有元素
    stack<int> stk;
    // 阶段性记录栈中的最小元素
    stack<int> minStk;

public:
    void push(int val) {
        stk.push(val);
        // 维护 minStk 栈顶为全栈最小元素
        if (minStk.empty() || val <= minStk.top()) {
            // 新插入的这个元素就是全栈最小的
            minStk.push(val);
        } else {
            // 插入的这个元素比较大
            minStk.push(minStk.top());
        }
    }

    void pop() {
        stk.pop();
        minStk.pop();
    }
}
```



```

int top() {
    return stk.top();
}

int getMin() {
    // minStk 栈顶为全栈最小元素
    return minStk.top();
}
};

// 优化版
class MinStack {
    // 记录栈中的所有元素
    stack<int> stk;
    // 阶段性记录栈中的最小元素
    stack<int> minStk;

public:
    void push(int val) {
        stk.push(val);
        // 维护 minStk 栈顶为全栈最小元素
        if (minStk.empty() || val <= minStk.top()) {
            // 新插入的这个元素就是全栈最小的
            minStk.push(val);
        }
    }

    void pop() {
        if (stk.top() == minStk.top()) {
            // 弹出的元素是全栈最小的
            minStk.pop();
        }
        stk.pop();
    }

    int top() {
        return stk.top();
    }

    int getMin() {
        // minStk 栈顶为全栈最小元素
        return minStk.top();
    }
};

```

895. 最大频率栈 | 力扣 | [LeetCode](#) |

设计一个类似堆栈的数据结构，将元素推入堆栈，并从堆栈中弹出出现频率最高的元素。

实现 FreqStack 类:

- FreqStack() 构造一个空的堆栈。
- void push(int val) 将一个整数 val 压入栈顶。
- int pop() 删除并返回堆栈中出现频率最高的元素。
 - 如果出现频率最高的元素不只一个，则移除并返回最接近栈顶的元素。

示例 1:

输入:

```
["FreqStack","push","push","push","push","push","push","pop","pop","pop","pop"],  
[5,7,5,7,4,5,0,0,0,0]
```

输出: [null,null,null,null,null,null,null,5,7,5,4]

解释:

```
FreqStack = new FreqStack();
```

```
freqStack.push(5); // 堆栈为 [5]
```

```
freqStack.push(7); // 堆栈是 [5,7]
```

```
freqStack.push(5); // 堆栈是 [5,7,5]
```

```
freqStack.push(7); // 堆栈是 [5,7,5,7]
```

```
freqStack.push(4); // 堆栈是 [5,7,5,7,4]
```

```
freqStack.push(5); // 堆栈是 [5,7,5,7,4,5]
```

```
freqStack.pop(); // 返回 5，因为 5 出现频率最高。堆栈变成 [5,7,5,7,4]。
```

```
freqStack.pop(); // 返回 7，因为 5 和 7 出现频率最高，但 7 最接近顶部。堆栈变成 [5,7,5,4]。
```

```
freqStack.pop(); // 返回 5，因为 5 出现频率最高。堆栈变成 [5,7,4]。
```

```
freqStack.pop(); // 返回 4，因为 4, 5 和 7 出现频率最高，但 4 是最接近顶部的。堆栈变成 [5,7]。
```

提示:

- $0 \leq \text{val} \leq 10^9$
- push 和 pop 的操作数不大于 $2 * 10^4$ 。
- 输入保证在调用 pop 之前堆栈中至少有一个元素。

这种设计数据结构的问题，主要是要搞清楚问题的难点在哪里，然后结合各种基本数据结构的特性，高效实现题目要求的 API。

那么，我们仔细思考一下 push 和 pop 方法，难点如下:

- 1、每次 pop 时，必须要知道频率最高的元素是什么。
- 2、如果频率最高的元素有多个，还得知哪个是最近 push 进来的元素是哪个。

为了实现上述难点，我们要做到以下几点:

- 1、肯定要有有一个变量 maxFreq 记录当前栈中最高的频率是多少。
- 2、我们得知道一个频率 freq 对应的元素有哪些，且这些元素要有时间顺序。

3、随着 pop 的调用，每个 val 对应的频率会变化，所以还得维持一个映射记录每个 val 对应的 freq。

综上，我们可以先实现 FreqStack 所需的数据结构：

```
class FreqStack {
    // 记录 FreqStack 中元素的最大频率
    int maxFreq = 0;
    // 记录 FreqStack 中每个 val 对应的出现频率，后文就称为 VF 表
    HashMap<Integer, Integer> valToFreq = new HashMap<>();
    // 记录频率 freq 对应的 val 列表，后文就称为 FV 表
    HashMap<Integer, Stack<Integer>> freqToVals = new HashMap<>();
}
```

其实这有点类似后文 [手把手实现 LFU 算法](#)，注意 freqToVals 中 val 列表用一个栈实现，如果一个 freq 对应的元素有多个，根据栈的特点，可以首先取出最近添加的元素。

具体看解法代码吧，要记住在 push 和 pop 方法中同时修改 maxFreq、VF 表、FV 表，否则容易出现 bug。

算法执行过程如下 GIF 所示：



```
#include <unordered_map>
#include <stack>
using namespace std;

class FreqStack {
    // 记录 FreqStack 中元素的最大频率
    int maxFreq = 0;
    // 记录 FreqStack 中每个 val 对应的出现频率，后文就称为 VF 表
```

```

unordered_map<int, int> valToFreq;
// 记录频率 freq 对应的 val 列表，后文就称为 FV 表
unordered_map<int, stack<int>> freqToVals;

public:
    void push(int val) {
        // 修改 VF 表: val 对应的 freq 加一
        int freq = valToFreq.count(val) ? valToFreq[val] + 1 : 1;
        valToFreq[val] = freq;
        // 修改 FV 表: 在 freq 对应的列表加上 val
        freqToVals[freq].push(val);
        // 更新 maxFreq
        maxFreq = max(maxFreq, freq);
    }

    int pop() {
        // 修改 FV 表: pop 出一个 maxFreq 对应的元素 v
        int v = freqToVals[maxFreq].top();
        freqToVals[maxFreq].pop();
        // 修改 VF 表: v 对应的 freq 减一
        valToFreq[v]--;
        // 更新 maxFreq
        if (freqToVals[maxFreq].empty()) {
            // 如果 maxFreq 对应的元素空了
            maxFreq--;
        }
        return v;
    }
};

```

括号类问题汇总

20. 有效的括号 | [力扣](#) | [LeetCode](#) |

给定一个只包括 '('，')'，'{'，'}'， '['，']' 的字符串 s，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

示例 1：

输入： s = "()"

输出： true

示例 2：

输入: `s = "()[]{}"`

输出: `true`

示例 3:

输入: `s = "("`

输出: `false`

提示:

- `1 <= s.length <= 104`
- `s` 仅由括号 `'()[]{}'` 组成

栈是一种先进后出的数据结构, 处理括号问题的时候尤其有用。

括号的有效性判断在笔试中和现实中都很常见, 比如说我们写的代码, 编辑器会检查括号是否正确闭合。而且我们的代码可能会包含三种括号 `[](){}`, 判断起来有一点难度。

解决这个问题之前, 我们先降低难度, 思考一下, **如果只有一种括号 `()`**, 应该如何判断字符串组成的括号是否有效呢?

假设字符串中只有圆括号, 如果想让括号字符串有效, 那么必须做到:

每个右括号 `)` 的左边必须有一个左括号 `(` 和它匹配。

比如说字符串 `(()))((` 中, 中间的两个右括号**左边**就没有左括号匹配, 所以这个括号组合是无效的。

那么根据这个思路, 我们可以写出算法:

```
boolean isValid(String str) {  
    // 待匹配的左括号数量  
    int left = 0;  
    for (int i = 0; i < str.length(); i++) {  
        if (str.charAt(i) == '(') {  
            left++;  
        } else {  
            // 遇到右括号  
            left--;  
        }  
  
        // 右括号太多  
        if (left == -1)  
            return false;  
    }  
    // 是否所有的左括号都被匹配了  
    return left == 0;  
}
```

如果只有圆括号，这样就能正确判断有效性。对于三种括号的情况，我一开始想模仿这个思路，定义三个变量 `left1`，`left2`，`left3` 分别处理每种括号，虽然要多写不少 `if else` 分支，但是似乎可以解决问题。

但实际上直接照搬这种思路是不行的，比如说只有一个括号的情况下 `(( ))` 是有效的，但是多种括号的情况下，`[( )]` 显然是无效的。

仅仅记录每种左括号出现的次数已经不能做出正确判断了，我们要加大存储的信息量，可以利用栈来模仿类似的思路。

我们这道题就用一个名为 `left` 的栈代替之前思路中的 `left` 变量，遇到左括号就入栈，遇到右括号就去栈中寻找最近的左括号，看是否匹配。

```
class Solution {
public:
    bool isValid(string str) {
        stack<char> left;
        for (char c : str) {
            if (c == '(' || c == '{' || c == '[') {
                // 字符 c 是左括号，入栈
                left.push(c);
            } else {
                // 字符 c 是右括号
                if (!left.empty() && leftOf(c) == left.top()) {
                    left.pop();
                } else {
                    // 和最近的左括号不匹配
                    return false;
                }
            }
        }
        // 是否所有的左括号都被匹配了
        return left.empty();
    }

private:
    char leftOf(char c) {
        if (c == '}') return '{';
        if (c == ')') return '(';
        return '[';
    }
};
```

921. 使括号有效的最少添加 | 力扣 | [LeetCode](#) |

只有满足下面几点之一，括号字符串才是有效的：

- 它是一个空字符串，或者

- 它可以被写成 `AB` (`A` 与 `B` 连接), 其中 `A` 和 `B` 都是有效字符串, 或者
- 它可以被写作 `(A)`, 其中 `A` 是有效字符串。

给定一个括号字符串 `s`, 在每一次操作中, 你都可以在字符串的任何位置插入一个括号

- 例如, 如果 `s = "())"`, 你可以插入一个开始括号为 `"(())"` 或结束括号为 `"(())"`。

返回 为使结果字符串 `s` 有效而必须添加的最少括号数。

示例 1:

输入: `s = "())"`

输出: 1

示例 2:

输入: `s = "((("`

输出: 3

提示:

- `1 <= s.length <= 1000`
- `s` 只包含 `'('` 和 `')'` 字符。

```
class Solution {
public:
    int minAddToMakeValid(string s) {
        // res 记录插入次数
        int res = 0;
        // need 变量记录右括号的需求量
        int need = 0;

        for (int i = 0; i < s.length(); i++) {
            if (s[i] == '(') {
                // 对右括号的需求 + 1
                need++;
            }

            if (s[i] == ')') {
                // 对右括号的需求 - 1
                need--;

                if (need == -1) {
                    need = 0;
                    // 需插入一个左括号
                    res++;
                }
            }
        }
    }
}
```

```

    }

    return res + need;
}
};

```

这段代码就是最终解法，核心思路是以左括号为基准，通过维护对右括号的需求数 `need`，来计算最小的插入次数。需要注意两个地方：

1、当 `need == -1` 的时候意味着什么？

因为只有遇到右括号 `)` 的时候才会 `need--`，`need == -1` 意味着右括号太多了，所以需要插入左括号。

比如说 `s = ")))"` 这种情况，需要插入 2 个左括号，使得 `s` 变成 `"()()"`，才是一个有效括号串。

2、算法为什么返回 `res + need`？

因为 `res` 记录的左括号的插入次数，`need` 记录了右括号的需求，当 `for` 循环结束后，若 `need` 不为 0，那么就意味着右括号还不够，需要插入。

比如说 `s = "))(("` 这种情况，插入 2 个左括号之后，还要再插入 1 个右括号，使得 `s` 变成 `"()()())"`，才是一个有效括号串。

下面是对应的用栈实现的版本，但是前者是 **$O(1)$ 空间复杂度** 的优化版本。

```

class Solution_Stack_921 {
public:
    int minAddToMakeValid(string s) {
        std::stack<char> st;
        int insertions = 0; // 记录需要插入的括号总数

        for (char c : s) {
            if (c == '(') {
                // 遇到左括号，入栈
                st.push(c);
            } else { // c == ')'
                if (!st.empty()) {
                    // 有左括号可以匹配，出栈
                    st.pop();
                } else {
                    // 栈为空，没有左括号可以匹配，说明这个右括号是多余的。
                    // 必须在它前面插入一个左括号 '('
                    insertions++;
                }
            }
        }
    }
};

```



```
        // 遍历结束后，栈中剩下的都是未匹配的左括号。
        // 每一个都需要插入一个右括号 ')' 来平衡。
        insertions += st.size();

        return insertions;
    }
};
```

以上就是这道题的思路，接下来我们看一道进阶题目，如果左右括号不是 1:1 配对，会出现什么问题呢？

1541. 平衡括号字符串的最少插入次数 | [力扣](#) | [LeetCode](#) |

给你一个括号字符串 `s`，它只包含字符 `'('` 和 `'('`。一个括号字符串被称为平衡的当它满足：

- 任何左括号 `'('` 必须对应两个连续的右括号 `'))'`。
- 左括号 `'('` 必须在对应的连续两个右括号 `'))'` 之前。

比方说 `"()"`，`"()())"` 和 `"(())())"` 都是平衡的，`")()"`，`"())"` 和 `"(())"` 都是不平衡的。

你可以在任意位置插入字符 `'('` 和 `')` 使字符串平衡。

请你返回让 `s` 平衡的最少插入次数。

示例 1：

输入：s = "(()))"

输出：1

解释：第二个左括号有与之匹配的两个右括号，但是第一个左括号只有一个右括号。我们需要在字符串结尾额外增加一个 `')` 使字符串变成平衡字符串 `"(())())"`。

示例 2：

输入：s = "())"

输出：0

解释：字符串已经平衡了。

示例 3：

输入：s = "))(())"

输出：3

解释：添加 `'('` 去匹配最开头的 `'))'`，然后添加 `'))'` 去匹配最后一个 `'('`。

示例 4：

输入：s = "((((("

输出：12

解释： 添加 12 个 ')' 得到平衡字符串。

示例 5：

输入： `s = "))))))"`

输出： 5

解释： 在字符串开头添加 4 个 '(' 并在结尾添加 1 个 ')'，字符串变成平衡字符串 "((((())))"。

提示：

- `1 <= s.length <= 10^5`
- `s` 只包含 '(' 和 ')'。

现在假设 1 个左括号需要匹配 2 个右括号才叫做有效的括号组合，那么给你输入一个字符串 `s`，请问你如何计算使得 `s` 有效的最小插入次数呢？

核心思路还是和刚才一样，通过一个 `need` 变量记录对右括号的需求数，根据 `need` 的变化来判断是否需要插入。

第一步，我们按照刚才的思路正确维护 `need` 变量：

```
int minInsertions(std::string s) {
    // need 记录需右括号的需求量
    int res = 0, need = 0;
    for (int i = 0; i < s.size(); i++) {
        // 一个左括号对应两个右括号
        if (s[i] == '(') {
            need += 2;
        }

        if (s[i] == ')') {
            need--;
        }
    }
    return res + need;
}
```

现在想一想，当 `need` 为什么值的时候，我们可以确定需要进行插入？

首先，类似第一题，当 `need == -1` 时，意味着我们遇到一个多余的右括号，显然需要插入一个左括号。

比如说当 `s = ")"`，我们肯定需要插入一个左括号让 `s = "()"`，但是由于一个左括号需要两个右括号，所以对右括号的需求量变为 1：

```
if (s[i] == ')') {
    need--;
```

```

// 说明右括号太多了
if (need == -1) {
    // 需要插入一个左括号
    res++;
    // 同时，对右括号的需求变为 1
    need = 1;
}
}

```

另外，当遇到左括号时，若对右括号的需求量为奇数，需要插入 1 个右括号。因为一个左括号需要两个右括号嘛，右括号的需求必须是偶数，这一点也是本题的难点。

所以遇到左括号时要做如下判断：

```

if (s[i] == '(') {
    need += 2;
    if (need % 2 == 1) {
        // 插入一个右括号
        res++;
        // 对右括号的需求减一
        need--;
    }
}
}

```

综上，我们可以写出正确的代码：

```

class Solution {
public:
    int minInsertions(string s) {
        // need 记录需右括号的需求量
        int res = 0, need = 0; // res指光标之前需要插入的括号数，need指之后

        for (int i = 0; i < s.length(); i++) {
            // 一个左括号对应两个右括号
            if (s[i] == '(') {
                need += 2;
                if (need % 2 == 1) { // 如()(时
                    // 插入一个右括号
                    res++;
                    need--;
                }
            }

            if (s[i] == ')') {
                need--;
                // 说明右括号太多了
                if (need == -1) {
                    // 需要插入一个左括号

```

```

        res++;
        // 同时，对右括号的需求变为 1
        need = 1;
    }
}

return res + need;
};

```

使用栈的版本，但是空间复杂度较高，前者为 $O(1)$ 后者为 $O(N)$

```

#include <string>
#include <stack>
#include <iostream>

class Solution_Stack_1541 {
public:
    int minInsertions(std::string s) {
        std::stack<char> st;
        int insertions = 0;
        int n = s.length();

        for (int i = 0; i < n; ++i) {
            if (s[i] == '(') {
                // 遇到左括号，入栈
                st.push('(');
            } else { // s[i] == ')'
                // 检查是否是连续的 ')'
                bool has_second_paren = (i + 1 < n) && (s[i + 1] == ')');

                if (has_second_paren) {
                    // 情况 A: 遇到了连续的 '))'
                    if (!st.empty()) {
                        // 栈中有 '(', 可以直接匹配 '))'
                        st.pop();
                    } else {
                        // 栈为空, '))' 是多余的, 需要插入一个 '('
                        insertions++;
                    }
                }
                // 跳过下一个 ')', 因为它已经被消耗了
                i++;
            } else {
                // 情况 B: 遇到了单个 ')'

                // 1. 必须插入一个 ')' 来凑成 '))'
                insertions++;
            }
        }
    }
};

```

```

        // 2. 现在我们有了 '))', 尝试匹配
        if (!st.empty()) {
            // 栈中有 '(', 成功匹配
            st.pop();
        } else {
            // 栈为空, 说明匹配的 '))' 是多余的,
            // 还需要插入一个 '(' 来平衡它们
            insertions++;
        }
    }

    // 遍历结束后, 栈中剩下的都是未匹配的 '('
    // 每个 '(' 都需要插入两个 ')'
    insertions += 2 * st.size();

    return insertions;
}
};

```

队列的经典习题

考察先进先出性质

队列常见考点主要是元素「先进先出」的顺序特性，比如维护队列内的元素在「时序上」的某些性质，下面是几道例题，队列充当「滑动窗口」的作用。

933. 最近的请求次数 | 力扣 | [LeetCode](#) | ●

写一个 `RecentCounter` 类来计算特定时间范围内最近的请求。

请你实现 `RecentCounter` 类：

- `RecentCounter()` 初始化计数器，请求数为 0。
- `int ping(int t)` 在时间 `t` 添加一个新请求，其中 `t` 表示以毫秒为单位的某个时间，并返回过去 3000 毫秒内发生的所有请求数（包括新请求）。确切地说，返回在 `[t-3000, t]` 内发生的请求数。

保证 每次对 `ping` 的调用都使用比之前更大的 `t` 值。

示例 1：

输入：

`["RecentCounter", "ping", "ping", "ping", "ping"]`

`[, [1], [100], [3001], [3002]]`

输出：

`[null, 1, 2, 3, 3]`

解释：

RecentCounter recentCounter = new RecentCounter();

recentCounter.ping(1); // requests = [1]，范围是 [-2999,1]，返回 1

recentCounter.ping(100); // requests = [1, 100]，范围是 [-2900,100]，返回 2

recentCounter.ping(3001); // requests = [1, 100, 3001]，范围是 [1,3001]，返回 3

recentCounter.ping(3002); // requests = [1, 100, 3001, 3002]，范围是 [2,3002]，返回 3

提示：

- $1 \leq t \leq 10^9$
- 保证每次对 ping 调用所使用的 t 值都 **严格递增**
- 至多调用 ping 方法 10⁴ 次

这题很简单，和 [346. 数据流中的移动平均值](#) 有点类似，要求动态维护队列中的元素。

```
class RecentCounter {
public:
    queue<int> q;

    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000) {
            // t 是递增的，所以可以从队头删除 3000 毫秒之前的请求
            q.pop();
        }

        return q.size();
    }
};
```

队列相关的设计题

1670. 设计前中后队列 | [力扣](#) | [LeetCode](#) |

请你设计一个队列，支持在前，中，后三个位置的 push 和 pop 操作。

请你完成 FrontMiddleBack 类：

- FrontMiddleBack() 初始化队列。
- void pushFront(int val) 将 val 添加到队列的 **最前面**。
- void pushMiddle(int val) 将 val 添加到队列的 **正中间**。
- void pushBack(int val) 将 val 添加到队里的 **最后面**。
- int popFront() 将 **最前面** 的元素从队列中删除并返回值，如果删除之前队列为空，那么返回 -1。
- int popMiddle() 将 **正中间** 的元素从队列中删除并返回值，如果删除之前队列为空，那么返回 -1。

- `int popBack()` 将 **最后面** 的元素从队列中删除并返回值，如果删除之前队列为空，那么返回 `-1`。

请注意当有 **两个** 中间位置的时候，选择靠前面的位置进行操作。比方说：

- 将 `6` 添加到 `[1, 2, 3, 4, 5]` 的中间位置，结果数组为 `[1, 2, **6**, 3, 4, 5]`。
- 从 `[1, 2, **3**, 4, 5, 6]` 的中间位置弹出元素，返回 `3`，数组变为 `[1, 2, 4, 5, 6]`。

示例 1：

输入：

`["FrontMiddleBackQueue", "pushFront", "pushBack", "pushMiddle", "pushMiddle", "popFront", "popMiddle", "popMiddle", "popBack", "popFront"]`

`1, [1], [2], [3], [4], [], [], [], [], [`

输出：

`[null, null, null, null, null, 1, 3, 4, 2, -1]`

解释：

```
FrontMiddleBackQueue q = new FrontMiddleBackQueue();
q.pushFront(1); // [1]
q.pushBack(2); // [1, 2]
q.pushMiddle(3); // [1, 3, 2]
q.pushMiddle(4); // [1, 4, 3, 2]
q.popFront(); // 返回 1 -> [4, 3, 2]
q.popMiddle(); // 返回 3 -> [4, 2]
q.popMiddle(); // 返回 4 -> [2]
q.popBack(); // 返回 2 -> []
q.popFront(); // 返回 -1 -> [] （队列为空）
```

提示：

- `1 <= val <= 109`
- 最多调用 `1000` 次 `pushFront`，`pushMiddle`，`pushBack`，`popFront`，`popMiddle` 和 `popBack`。

这题有点难度，主要是细节不好把控。常规的队列只能在首尾进行操作，想在中间操作队列，需要在底层把队列切分成 `left`，`right` 两个列表，但这里的细节问题就是元素为奇数时两个链表中元素的分配问题。

```
class FrontMiddleBackQueue {
    // 用两个列表表示队列的左右两部分，以便从中间操作元素
    deque<int> left;
    deque<int> right;
    // 如果是奇数个元素，维护左边少右边多，所以：
```

```
// 1、如果有偶数个元素时，pushMiddle 优先向右边添加
// 2、如果有奇数个元素时，popMiddle 优先从右边删除
// 3、如果只有 1 个元素，popFront 的时候，要去右边删除
// 要把以上三个特点写到代码里，才能保证细节不出错
```

```
// 维护左边少右边多的状态，每次增删元素之后都要执行一次
```

```
void balance() {
    // 右边最多比左边多一个元素
    if (right.size() > left.size() + 1) {
        // 右边多，匀一个给左边
        left.push_back(right.front());
        right.pop_front();
    }
    if (left.size() > right.size()) {
        // 左边多，匀一个给右边
        right.push_front(left.back());
        left.pop_back();
    }
}
```

```
public:
```

```
void pushFront(int val) {
    left.push_front(val);
    balance();
}
```

```
void pushMiddle(int val) {
    if (size() % 2 == 0) {
        // 如果有偶数个元素时，pushMiddle 优先向右边添加
        right.push_front(val);
    } else {
        left.push_back(val);
    }
    balance();
}
```

```
void pushBack(int val) {
    right.push_back(val);
    balance();
}
```

```
int popFront() {
    if (size() == 0) {
        return -1;
    }
    if (size() == 1) {
        // 如果只有 1 个元素，popFront 的时候，要去右边删除
        int e = right.front();
        right.pop_front();
        return e;
    }
}
```



```

    }
    int e = left.front();
    left.pop_front();
    balance();
    return e;
}

int popMiddle() {
    if (size() == 0) {
        return -1;
    }
    int e;
    if (size() % 2 == 0) {
        e = left.back();
        left.pop_back();
    } else {
        // 如果有奇数个元素时，popMiddle 优先从右边删除
        e = right.front();
        right.pop_front();
    }
    balance();
    return e;
}

int popBack() {
    if (size() == 0) {
        return -1;
    }
    int e = right.back();
    right.pop_back();
    balance();
    return e;
}

int size() {
    return left.size() + right.size();
}
};

```

2073. 买票需要的时间 | 力扣 | [LeetCode](#) | ●

有 n 个人前来排队买票，其中第 0 人站在队伍 **最前方**，第 $(n - 1)$ 人站在队伍 **最后方**。

给你一个下标从 **0** 开始的整数数组 `tickets`，数组长度为 n ，其中第 i 人想要购买的票数为 `tickets[i]`。

每个人买票都需要用掉 **恰好 1 秒**。一个人 **一次只能买一张票**，如果需要购买更多票，他必须走到 **队尾** 重新排队（**瞬间** 发生，不计时间）。如果一个人没有剩下需要买的票，那他将会 **离开** 队伍。

返回位于位置 `k`（下标从 `0` 开始）的人完成买票需要的时间（以秒为单位）。

示例 1:

输入: `tickets = [2,3,2]`, `k = 2`

输出: `6`

解释:

- 第一轮，队伍中的每个人都买到一张票，队伍变为 `[1, 2, 1]`。
- 第二轮，队伍中的每个都又都买到一张票，队伍变为 `[0, 1, 0]`。
位置 `2` 的人成功买到 `2` 张票，用掉 $3 + 3 = 6$ 秒。

示例 2:

输入: `tickets = [5,1,1,1]`, `k = 0`

输出: `8`

解释:

- 第一轮，队伍中的每个人都买到一张票，队伍变为 `[4, 0, 0, 0]`。
- 接下来的 `4` 轮，只有位置 `0` 的人在买票。
位置 `0` 的人成功买到 `5` 张票，用掉 $4 + 1 + 1 + 1 + 1 = 8$ 秒。

提示:

- `n == tickets.length`
- `1 <= n <= 100`
- `1 <= tickets[i] <= 100`
- `0 <= k < n`

既然是排队问题，你用一个队列模拟整个买票过程，然后数一数过了多少秒就行了，不过时间空间复杂度就高了。

稍微思考一下可以想到更高效的方式：

首先，第 `k` 个人离开的时间，其实就是从开始到这个人买完票之后，卖出的总票数。

那么第 `k` 个人买完票之后，总共卖了多少票呢？

排在 `k` 之前的人最多买了 `tickets[k]` 张票；而排在 `k` 之后的人最多买了 `tickets[k] - 1` 张票。为什么说「最多」呢，因为有的人可能需要的票比较少，买完就直接走了。

综上，思路就出来了，看代码理解吧。

```
class Solution {
public:
    int timeRequiredToBuy(vector<int>& tickets, int k) {
        int res = 0;
        for (int i = 0; i < tickets.size(); i++) {
```

```
        if (i <= k) {
            // 前面的人最多买了 tickets[k] 张票
            res += min(tickets[k], tickets[i]);
        } else {
            // 后面的人最多买了 tickets[k] - 1 张票
            res += min(tickets[k] - 1, tickets[i]);
        }
    }
    return res;
}
};
```

单调栈

单调栈模板

栈 (stack) 是很简单的一种数据结构，先进后出的逻辑顺序，符合某些问题的特点，比如说函数调用栈。单调栈实际上就是栈，只是利用了一些巧妙的逻辑，使得每次新元素入栈后，栈内的元素都保持有序（单调递增或单调递减）。

听起来有点像堆 (heap)？不是的，单调栈用途不太广泛，只处理一类典型的问题，比如「下一个更大元素」，「上一个更小元素」等。本文讲解单调栈的算法模版解决「下一个更大元素」相关问题，并且探讨处理「循环数组」的策略。

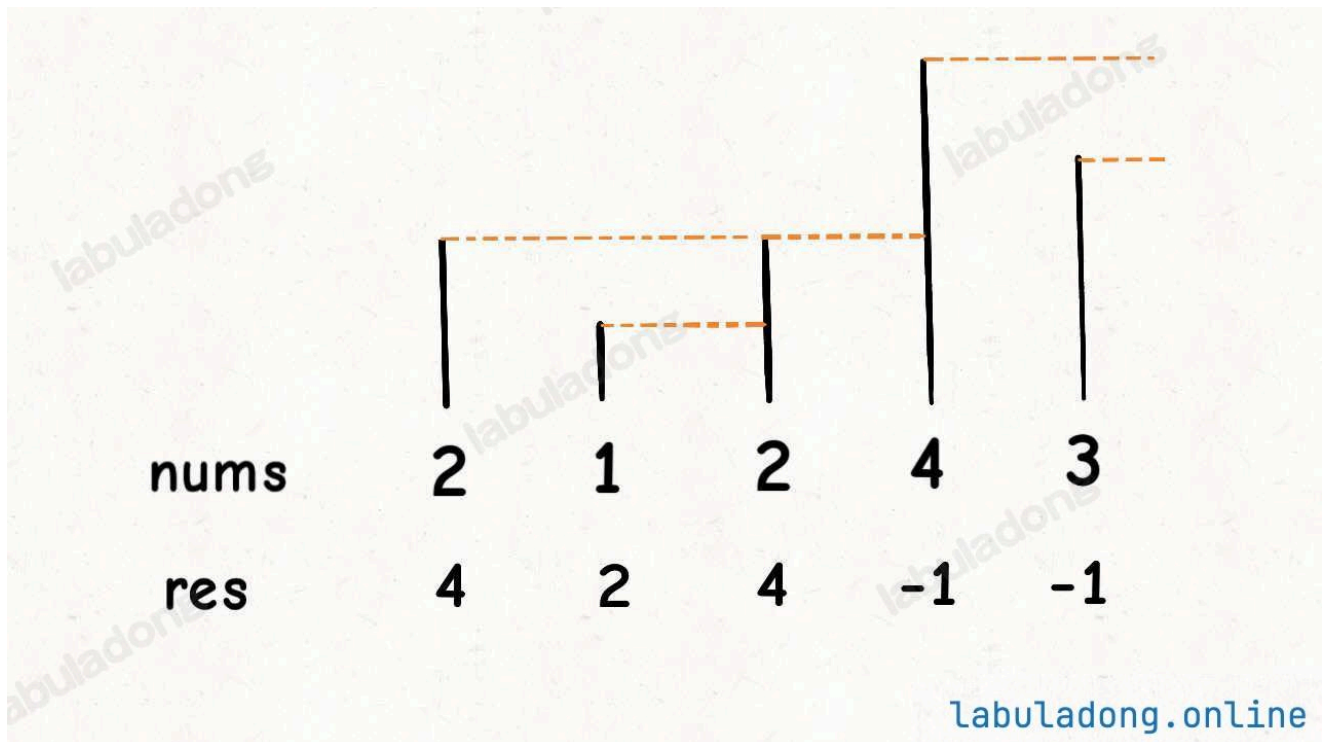
现在给你出这么一道题：输入一个数组 `nums`，请你返回一个等长的结果数组，结果数组中对应索引存储着下一个更大元素，如果没有更大的元素，就存 -1。函数签名如下：

```
vector<int> calculateGreaterElement(vector<int>& nums);
```

比如说，输入一个数组 `nums = [2,1,2,4,3]`，你返回数组 `[4,2,4,-1,-1]`。因为第一个 2 后面比 2 大的数是 4；1 后面比 1 大的数是 2；第二个 2 后面比 2 大的数是 4；4 后面没有比 4 大的数，填 -1；3 后面没有比 3 大的数，填 -1。

这道题的暴力解法很好想到，就是对每个元素后面都进行扫描，找到第一个更大的元素就行了。但是暴力解法的时间复杂度是 $O(n^2)$ 。

这个问题可以这样抽象思考：把数组的元素想象成并列站立的人，元素大小想象成人的身高。这些人面对你站成一列，如何求元素「2」的下一个更大元素呢？很简单，如果能够看到元素「2」，那么他后面可见的第一个人就是「2」的下一个更大元素，因为比「2」小的元素身高不够，都被「2」挡住了，第一个露出来的就是答案。



这个情景很好理解吧？带着这个抽象的情景，先看下代码。

```
vector<int> calculateGreaterElement(vector<int>& nums) {  
    int n = nums.size();  
    // 存放答案的数组  
    vector<int> res(n);  
    stack<int> s;  
    // 倒着往栈里放  
    for (int i = n - 1; i >= 0; i--) {  
        // 判定个子高矮  
        while (!s.empty() && s.top() <= nums[i]) {  
            // 矮个起开，反正也被挡着了。。。  
            s.pop();  
        }  
        // nums[i] 身后的更大元素  
        res[i] = s.empty() ? -1 : s.top();  
        s.push(nums[i]);  
    }  
    return res;  
}
```

这就是单调栈解决问题的模板。for 循环要从后往前扫描元素，因为我们借助的是栈的结构，倒着入栈，其实是正着出栈。while 循环是把两个「个子高」元素之间的元素排除，因为他们的存在没有意义，前面挡着个「更高」的元素，所以他们不可能被作为后续进来的元素的下一个更大元素了。

这个算法的时间复杂度不是那么直观，如果你看到 for 循环嵌套 while 循环，可能认为这个算法的复杂度也是 $O(n^2)$ ，但是实际上这个算法的复杂度只有 $O(n)$ 。

分析它的时间复杂度，要从整体来看：总共有 n 个元素，每个元素都被 `push` 入栈了一次，而最多会被 `pop` 一次，没有任何冗余操作。所以总的计算规模是和元素规模 n 成正比的，也就是 $O(n)$ 的复杂度。

问题变形

单调栈的代码实现比较简单，下面来看一些具体题目。

496. 下一个更大元素 I

首先来一个简单的变形，力扣第 496 题「[下一个更大元素 I](#)」：

`nums1` 中数字 x 的 **下一个更大元素** 是指 x 在 `nums2` 中对应位置 **右侧** 的 **第一个** 比 x 大的元素。

给你两个 **没有重复元素** 的数组 `nums1` 和 `nums2`，下标从 **0** 开始计数，其中 `nums1` 是 `nums2` 的子集。

对于每个 $0 \leq i < \text{nums1.length}$ ，找出满足 `nums1[i] == nums2[j]` 的下标 j ，并且在 `nums2` 确定 `nums2[j]` 的 **下一个更大元素**。如果不存在下一个更大元素，那么本次查询的答案是 `-1`。

返回一个长度为 `nums1.length` 的数组 `ans` 作为答案，满足 `ans[i]` 是如上所述的 **下一个更大元素**。

示例 1：

输入： `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`.

输出： `[-1,3,-1]`

解释： `nums1` 中每个值的下一个更大元素如下所述：

- 4，用加粗斜体标识，`nums2 = [1,3,4,2]`。不存在下一个更大元素，所以答案是 `-1`。
- 1，用加粗斜体标识，`nums2 = [1,3,4,2]`。下一个更大元素是 3。
- 2，用加粗斜体标识，`nums2 = [1,3,4,2]`。不存在下一个更大元素，所以答案是 `-1`。

示例 2：

输入： `nums1 = [2,4]`, `nums2 = [1,2,3,4]`.

输出： `[3,-1]`

解释： `nums1` 中每个值的下一个更大元素如下所述：

- 2，用加粗斜体标识，`nums2 = [1,2,3,4]`。下一个更大元素是 3。
- 4，用加粗斜体标识，`nums2 = [1,2,3,4]`。不存在下一个更大元素，所以答案是 `-1`。

提示：

- `1 <= nums1.length <= nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 104`

- nums1 和 nums2 中所有整数 **互不相同**
- nums1 中的所有整数同样出现在 nums2 中

进阶： 你可以设计一个时间复杂度为 $O(\text{nums1.length} + \text{nums2.length})$ 的解决方案吗？

题目来源：[力扣 496. 下一个更大元素 I](#)。

这道题给你输入两个数组 nums1 和 nums2，让你求 nums1 中的元素在 nums2 中的下一个更大元素，函数签名如下：

```
vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2);
```

其实和把我们刚才的代码改一改就可以解决这道题了，因为题目说 nums1 是 nums2 的子集，那么我们先在 nums2 中每个元素的下一个更大元素算出来存到一个映射里，然后再让 nums1 中的元素去查表即可：

```
#include <vector>
#include <stack>
#include <unordered_map>

class Solution {
public:
    std::vector<int> nextGreaterElement(std::vector<int>& nums1,
std::vector<int>& nums2) {
        // 记录 nums2 中每个元素的下一个更大元素
        std::vector<int> greater = nextGreaterElement(nums2);
        // 转化成映射：元素 x -> x 的下一个最大元素
        std::unordered_map<int, int> greaterMap;
        for (int i = 0; i < nums2.size(); i++) {
            greaterMap[nums2[i]] = greater[i];
        }
        // nums1 是 nums2 的子集，所以根据 greaterMap 可以得到结果
        std::vector<int> res(nums1.size());
        for (int i = 0; i < nums1.size(); i++) {
            res[i] = greaterMap[nums1[i]];
        }
        return res;
    }

    // 计算 nums 中每个元素的下一个更大元素
    std::vector<int> nextGreaterElement(std::vector<int>& nums) {
        int n = nums.size();
        // 存放答案的数组
        std::vector<int> res(n);
        std::stack<int> s;
        // 倒着往栈里放
        for (int i = n - 1; i >= 0; i--) {
            // 判定个子高矮
```

```

        while (!s.empty() && s.top() <= nums[i]) {
            // 矮个起开，反正也被挡着了。。。
            s.pop();
        }
        // nums[i] 身后的下一个更大元素
        res[i] = s.empty() ? -1 : s.top();
        s.push(nums[i]);
    }
    return res;
}
};

```

739. 每日温度

再看看力扣第 739 题「[每日温度](#)」：

给你一个数组 `temperatures`，这个数组存放的是近几天的天气气温，你返回一个等长的数组，计算：对于每一天，你还要至少等多少天才能等到一个更暖和的气温；如果等不到那一天，填 0。函数签名如下：

```
vector<int> dailyTemperatures(vector<int>& temperatures);
```

比如说给你输入 `temperatures = [73,74,75,71,69,76]`，你返回 `[1,1,3,2,1,0]`。因为第一天 73 华氏度，第二天 74 华氏度，比 73 大，所以对于第一天，只要等一天就能等到一个更暖和的气温，后面的同理。

这个问题本质上也是找下一个更大元素，只不过现在不是问你下一个更大元素的值是多少，而是问你当前元素距离下一个更大元素的索引距离而已。

相同的思路，直接调用单调栈的算法模板，稍作改动就可以，直接上代码吧：

```

class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();
        vector<int> res(n, 0);
        // 这里放元素索引，而不是元素
        stack<int> s;
        // 单调栈模板
        for (int i = n - 1; i >= 0; i--) {
            while (!s.empty() && temperatures[s.top()] <= temperatures[i]) {
                s.pop();
            }
            // 得到索引间距
            res[i] = s.empty() ? 0 : (s.top() - i);
            // 将索引入栈，而不是元素
            s.push(i);
        }
        return res;
    }
};

```

```
    }  
};
```

单调栈讲解完毕，下面开始另一个重点：如何处理「循环数组」。

如何处理环形数组

同样是求下一个更大元素，现在假设给你的数组是个环形的，如何处理？力扣第 503 题「[下一个更大元素 II](#)」就是这个问题：输入一个「环形数组」，请你计算其中每个元素的下一个更大元素。

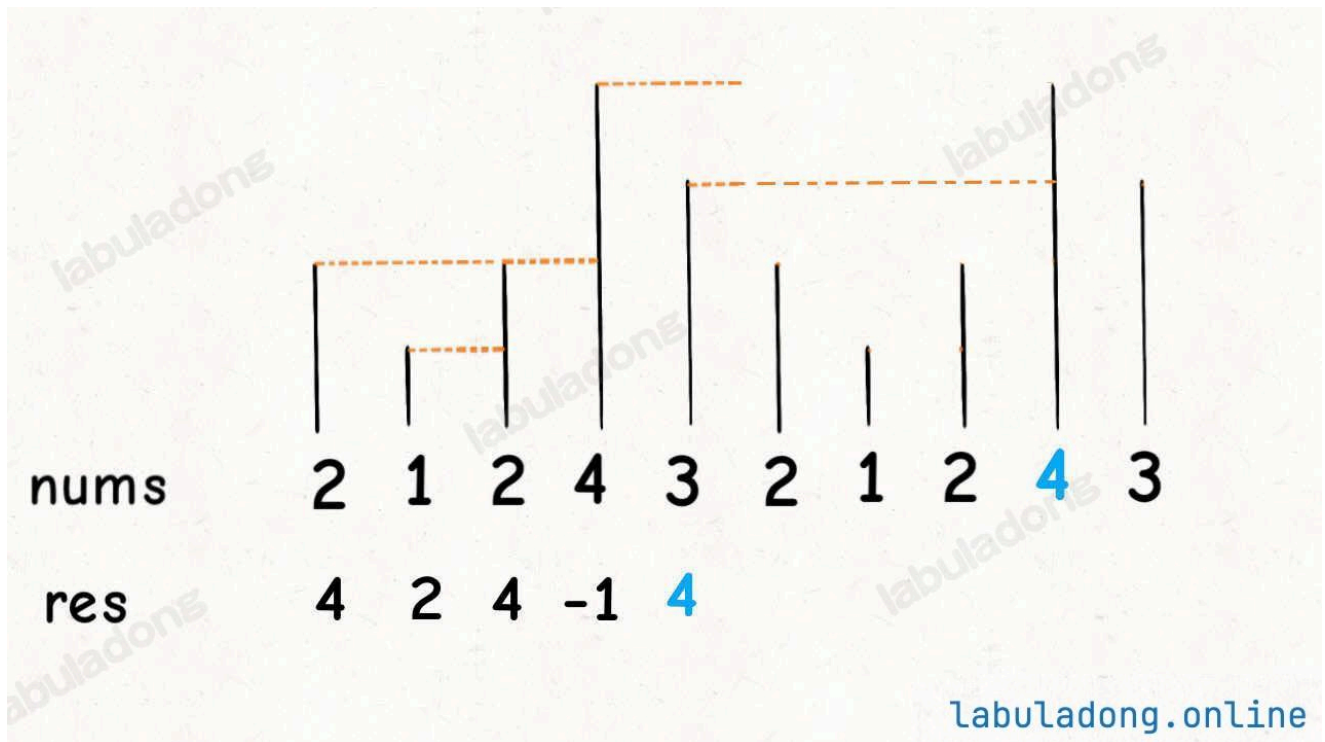
比如输入 `[2,1,2,4,3]`，你应该返回 `[4,2,4,-1,4]`，因为拥有了环形属性，**最后一个元素 3 绕了一圈后找到了比自己大的元素 4**。

如果你看过基础知识章节的[环形数组技巧](#)应该比较熟悉，我们一般是通过 % 运算符求模（余数），来模拟环形特效：

```
int main(){  
    int arr[5] = {1, 2, 3, 4, 5};  
    int n = sizeof(arr) / sizeof(arr[0]), index = 0;  
    while (true) {  
        // 在环形数组中转圈  
        cout << arr[index % n] << endl;  
        index++;  
    }  
    return 0;  
}
```

这个问题肯定还是要用单调栈的解题模板，但难点在于，比如输入是 `[2,1,2,4,3]`，对于最后一个元素 3，如何找到元素 4 作为下一个更大元素。

对于这种需求，常用套路就是将数组长度翻倍：



这样，元素 3 就可以找到元素 4 作为下一个更大元素了，而且其他的元素都可以被正确地计算。

有了思路，最简单的实现方式当然可以把这个双倍长度的数组构造出来，然后套用算法模板。但是，我们可以不用构造新数组，而是利用循环数组的技巧来模拟数组长度翻倍的效果。直接看代码吧：

```
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        int n = nums.size();
        vector<int> res(n);
        stack<int> s;
        // 数组长度加倍模拟环形数组
        for (int i = 2 * n - 1; i >= 0; i--) {
            // 索引 i 要求模，其他的和模板一样
            while (!s.empty() && s.top() <= nums[i % n]) {
                s.pop();
            }
            res[i % n] = s.empty() ? -1 : s.top();
            s.push(nums[i % n]);
        }
        return res;
    }
};
```

这样，就可以巧妙解决环形数组的问题，时间复杂度 $O(N)$ 。

最后提出一些问题吧，本文提供的单调栈模板是 `nextGreaterElement` 函数，可以计算每个元素的下一个更大元素，但如果题目让你计算上一个更大元素，或者计算上一个更大或相等的

元素，应该如何修改对应的模板呢？而且在实际应用中，题目不会直接让你计算下一个（上一个）更大（小）的元素，你如何把问题转化成单调栈相关的问题呢？

单调栈模板的变体

下一个更大的元素

上篇文章 [单调栈的实现](#) 带你使用单调栈解决了「下一个更大元素」的问题，比如下面这个例子：

```
输入：nums = [1, 3, 2, 4, 4]
下一个更大元素：[3, 4, 4, -1, -1]
```

我们实现了这样一个函数解决这个问题：

```
// 计算 nums 中每个元素的下一个更大元素
int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    // 存放答案的数组
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    // 因为是求 nums[i] 后面的元素，所以倒着往栈里放
    for (int i = n - 1; i >= 0; i--) {
        // 删掉 nums[i] 后面较小的元素
        while (!stk.isEmpty() && stk.peek() <= nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 身后的更大元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
    return res;
}
```

下一个更大或相等的元素

本文给出这个问题的一些变体，比如说让你计算 `nums[i]` 的下一个大于等于 `nums[i]` 的元素怎么算？比如下面这个例子：

```
输入：nums = [1, 3, 2, 4, 4]
返回：res = [3, 4, 4, 4, -1]
```

其实很简单，把上面这段代码中 `while` 循环的 `<=` 号改成 `<` 号即可：

```
// 计算 nums 中每个元素的下一个更大或相等的元素
int[] nextGreaterOrEqualElement(int[] nums) {
```

```

int n = nums.length;
int[] res = new int[n];
Stack<Integer> stk = new Stack<>();
for (int i = n - 1; i >= 0; i--) {
    // 把这里改成 < 号
    while (!stk.isEmpty() && stk.peek() < nums[i]) {
        stk.pop();
    }
    // 现在栈顶就是 nums[i] 身后的大于等于 nums[i] 的元素
    res[i] = stk.isEmpty() ? -1 : stk.peek();
    stk.push(nums[i]);
}
return res;
}

```

下一个更小的元素

再变一变，如果让你计算 `nums[i]` 的下一个小于 `nums[i]` 的元素，怎么算？比如下面这个例子：

```

输入: nums = [8, 4, 6, 6, 3]
返回: res = [4, 3, 3, 3, -1]

```

也很简单，把之前实现的 `nextGreaterElement` 中 `while` 循环的 `<=` 条件改成 `>=` 条件即可得出下一个更小的元素：

```

// 计算 nums 中每个元素的下一个更小的元素
int[] nextLessElement(int[] nums) {
    int n = nums.length;
    // 存放答案的数组
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    // 倒着往栈里放
    for (int i = n - 1; i >= 0; i--) {
        // 删掉 nums[i] 后面较大的元素
        while (!stk.isEmpty() && stk.peek() >= nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 身后的更小元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
    return res;
}

```

下一个更小或相等的元素

如果让你计算 `nums[i]` 的下一个小于或等于 `nums[i]` 的元素，怎么算？比如下面这个例子：

```
输入: nums = [8, 4, 6, 6, 3]
返回: res = [4, 3, 6, 3, -1]
```

类似的，把 `nextLessElement` 函数的 `while` 循环中的 `>=` 改成 `>` 即可：

```
// 计算 nums 中每个元素的下一个更小或相等的元素
int[] nextLessOrEqualElement(int[] nums) {
    int n = nums.length;
    // 存放答案的数组
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    // 倒着往栈里放
    for (int i = n - 1; i >= 0; i--) {
        // 删掉 nums[i] 后面较大的元素
        while (!stk.isEmpty() && stk.peek() > nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 身后的更小或相等元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
    return res;
}
```

上一个更大元素

之前的 4 个例子都是计算 `nums[i]` 的下一个更大/更小元素，现在请你计算 `nums[i]` 的上一个更大元素，你会不会？比如这个例子：

```
输入: nums = [8, 7, 6, 7]
返回: res = [-1, 8, 7, 8]
```

注意之前我们的 `for` 循环都是从数组的尾部开始往栈里添加元素，这样栈顶元素就是 `nums[i]` 之后的元素。所以只要我们从数组的头部开始往栈里添加元素，栈顶的元素就是 `nums[i]` 之前的元素，即可计算 `nums[i]` 的上一个更大元素。

代码实现如下：

```
// 计算 nums 中每个元素的上一个更大元素
int[] prevGreaterElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
```

```

// 因为是求 nums[i] 前面的元素，所以正着往栈里放
for (int i = 0; i < n; i++) {
    // 删掉 nums[i] 前面较小的元素
    while (!stk.isEmpty() && stk.peek() <= nums[i]) {
        stk.pop();
    }
    // 现在栈顶就是 nums[i] 前面的更大元素
    res[i] = stk.isEmpty() ? -1 : stk.peek();
    stk.push(nums[i]);
}
return res;
}

```

类似之前的几种实现，基于这个函数还可以求出 `nums[i]` 的上一个更大或相等的元素、上一个更小的元素、上一个更小或相等的元素，只要改一改 `while` 循环的符号即可，下面一一列出具体实现。

上一个更大或相等的元素

举例：

```

输入：nums = [8, 7, 6, 7]
返回：res = [-1, 8, 7, 8]

```

代码实现：

```

// 计算 nums 中每个元素的上一个更大或相等元素
int[] prevGreaterOrEqualElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    for (int i = 0; i < n; i++) {
        // 注意不等号
        while (!stk.isEmpty() && stk.peek() < nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 前面的更大或相等元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
    return res;
}

```

上一个更小的元素

举例：

输入: `nums = [3, 6, 6, 5]`

返回: `res = [-1, 3, 3, 3]`

代码实现:

```
// 计算 nums 中每个元素的上一个更小的元素
int[] prevLessElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    for (int i = 0; i < n; i++) {
        // 把 nums[i] 之前的较大元素删除
        while (!stk.isEmpty() && stk.peek() >= nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 前面的更小元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
    return res;
}
```

上一个更小或相等的元素

举例:

输入: `nums = [3, 6, 6, 5]`

返回: `res = [-1, 3, 6, 3]`

代码实现:

```
// 计算 nums 中每个元素的上一个更小或相等元素
int[] prevLessOrEqualElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Stack<Integer> stk = new Stack<>();
    for (int i = 0; i < n; i++) {
        // 注意不等号
        while (!stk.isEmpty() && stk.peek() > nums[i]) {
            stk.pop();
        }
        // 现在栈顶就是 nums[i] 前面的更小或相等元素
        res[i] = stk.isEmpty() ? -1 : stk.peek();
        stk.push(nums[i]);
    }
}
```

```
    return res;
}
```

至此，单调栈的几种标准模板就列举完了，他们之间有很多共性，理解性记忆就好，不用死记硬背。在实际算法题中不会直接考察你这些标准场景，但是稍加思考就可以抽象成这些标准场景。

下面带大家做几道习题练习一下单调栈模板的使用。

习题

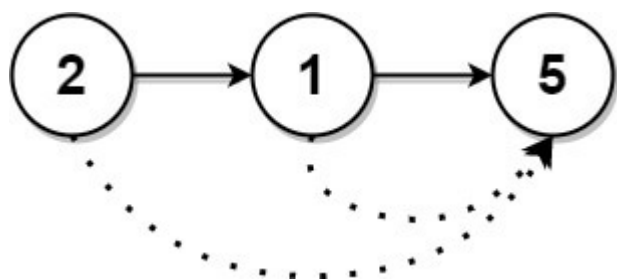
1019. 链表中的下一个更大节点

给定一个长度为 n 的链表 `head`

对于列表中的每个节点，查找下一个 **更大节点** 的值。也就是说，对于每个节点，找到它旁边的第一个节点的值，这个节点的值 **严格大于** 它的值。

返回一个整数数组 `answer`，其中 `answer[i]` 是第 i 个节点(从1开始)的下一个更大的节点的值。如果第 i 个节点没有下一个更大的节点，设置 `answer[i] = 0`。

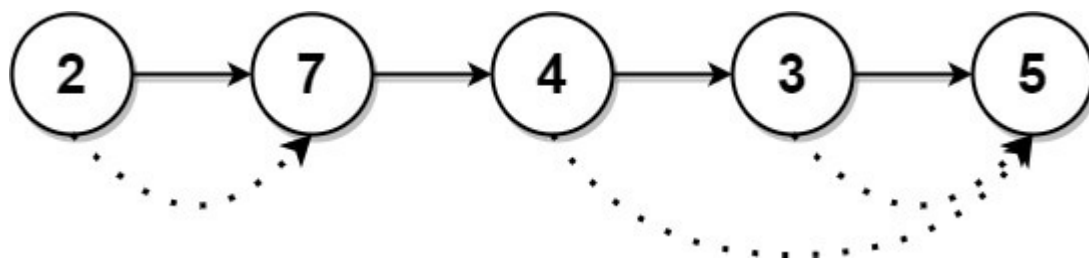
示例 1:



输入: `head = [2,1,5]`

输出: `[5,5,0]`

示例 2:



输入: `head = [2,7,4,3,5]`

输出: `[7,0,5,5,0]`

提示:

- 链表中节点数为 n
- $1 \leq n \leq 10^4$

- `1 <= Node.val <= 109`

题目来源：[力扣 1019. 链表中的下一个更大节点](#)。

这道题输入的是一条单链表，我们把它转化成数组，方便用索引访问即可直接套用 [单调栈模板](#) 中的 `nextGreaterElement` 函数逻辑。

```
#include <vector>
#include <stack>

using namespace std;

class Solution {
public:
    vector<int> nextLargerNodes(ListNode* head) {
        // 把单链表转化成数组，方便通过索引访问
        vector<int> nums;
        for (ListNode* p = head; p != nullptr; p = p->next) {
            nums.push_back(p->val);
        }
        // 存放答案的数组
        vector<int> res(nums.size());
        stack<int> stk;
        // 单调栈模板，求下一个更大元素，从后往前遍历
        for (int i = nums.size() - 1; i >= 0; i--) {
            while (!stk.empty() && stk.top() <= nums[i]) {
                stk.pop();
            }
            // 本题要求没有下一个更大元素时返回 0
            res[i] = stk.empty() ? 0 : stk.top();
            stk.push(nums[i]);
        }
        return res;
    }
};
```

1944. 队列中可以看到的人数

有 n 个人排成一个队列，**从左到右** 编号为 0 到 $n - 1$ 。给你以一个整数数组 `heights`，每个整数 **互不相同**，`heights[i]` 表示第 i 个人的高度。

一个人能 **看到** 他右边另一个人的条件是这两人之间的所有人都比他们两人 **矮**。更正式的，第 i 个人能看到第 j 个人的条件是 $i < j$ 且 $\min(\text{heights}[i], \text{heights}[j]) > \max(\text{heights}[i+1], \text{heights}[i+2], \dots, \text{heights}[j-1])$ 。

请你返回一个长度为 n 的数组 `answer`，其中 `answer[i]` 是第 i 个人在他右侧队列中能 **看到** 的人数。

示例 1:



输入: heights = [10,6,8,5,11,9]

输出: [3,1,2,1,1,0]

解释:

第 0 个人能看到编号为 1 , 2 和 4 的人。

第 1 个人能看到编号为 2 的人。

第 2 个人能看到编号为 3 和 4 的人。

第 3 个人能看到编号为 4 的人。

第 4 个人能看到编号为 5 的人。

第 5 个人谁也看不到因为他右边没人。

示例 2:

输入: heights = [5,1,2,3,10]

输出: [4,1,1,1,0]

提示:

- `n == heights.length`
- `1 <= n <= 105`
- `1 <= heights[i] <= 105`
- heights 中所有数 **互不相同**。

题目来源: [力扣 1944. 队列中可以看到的人数](#)。

这道题显然要用到 [单调栈技巧](#): 靠左的高个子可以把靠右相邻的矮个子都「挤掉」, 相当于计算下一个更大元素, 即 [单调栈的几种模板实现](#) 中的 `nextGreaterElement` 函数。

只不过这道题不是问你下一个更大元素是多少, 而是问你当前元素和下一个更大元素之间的元素个数, 直接看解法代码吧。

```
#include <vector>
#include <stack>
```

```

using namespace std;

class Solution {
public:
    vector<int> canSeePersonsCount(vector<int>& heights) {
        int n = heights.size();
        vector<int> res(n);
        // int[] 记录 {身高, 小于等于该身高的人数} 二元组
        stack<int> stk;
        for (int i = n - 1; i >= 0; i--) {
            // 记录右侧比自己矮的人
            int count = 0;
            // 单调栈模板, 计算下一个更大或相等元素 (身高)
            while (!stk.empty() && heights[i] > stk.top()) {
                stk.pop();
                count++;
            }
            // 不仅可以看到比自己矮的人, 如果后面存在更高的的人, 也可以看到这个高人
            res[i] = stk.empty() ? count : count + 1;
            stk.push(heights[i]);
        }
        return res;
    }
};

```

1475. 商品折扣后的最终价格

给你一个数组 `prices` , 其中 `prices[i]` 是商店里第 `i` 件商品的价格。

商店里正在进行促销活动, 如果你要买第 `i` 件商品, 那么你可以得到与 `prices[j]` 相等的折扣, 其中 `j` 是满足 `j > i` 且 `prices[j] <= prices[i]` 的 **最小下标** , 如果没有满足条件的 `j` , 你将没有任何折扣。

请你返回一个数组, 数组中第 `i` 个元素是折扣后你购买商品 `i` 最终需要支付的价格。

示例 1:

输入: `prices = [8,4,6,2,3]`

输出: `[4,2,4,2,3]`

解释:

商品 0 的价格为 `price[0]=8` , 你将得到 `prices[1]=4` 的折扣, 所以最终价格为 $8 - 4 = 4$ 。

商品 1 的价格为 `price[1]=4` , 你将得到 `prices[3]=2` 的折扣, 所以最终价格为 $4 - 2 = 2$ 。

商品 2 的价格为 `price[2]=6` , 你将得到 `prices[3]=2` 的折扣, 所以最终价格为 $6 - 2 = 4$ 。

商品 3 和 4 都没有折扣。

示例 2:

输入: `prices = [1,2,3,4,5]`

输出: `[1,2,3,4,5]`

解释： 在这个例子中，所有商品都没有折扣。

示例 3：

输入： prices = [10,1,1,6]

输出： [9,0,1,6]

提示：

- `1 <= prices.length <= 500`
- `1 <= prices[i] <= 10^3`

题目来源：[力扣 1475. 商品折扣后的最终价格](#)。

这道题就用到了 [单调栈的几种模板实现](#) 中讲到的一个单调栈模板：计算下一个更小或相等的元素。我是为了运用模板，所以把解法分成了两个函数，效率可能会降低一些，你完全可以优化这个解法的形式，使之更高效。

```
#include <vector>
#include <stack>

class Solution {
public:
    std::vector<int> finalPrices(std::vector<int>& prices) {
        int n = prices.size();
        std::vector<int> res(n);
        // 下一个小于等于 price[i] 的价格就是优惠券折扣
        std::vector<int> nextElement = nextLessOrEqualElement(prices);
        for (int i = 0; i < n; i++) {
            // 如果存在优惠券，则减少相应的价格
            if (nextElement[i] != -1) {
                res[i] = prices[i] - nextElement[i];
            } else {
                res[i] = prices[i];
            }
        }
        return res;
    }

    // 单调栈模板：计算 nums 中每个元素的下一个更小或相等的元素
    std::vector<int> nextLessOrEqualElement(std::vector<int>& nums) {
        int n = nums.size();
        // 存放答案的数组
        std::vector<int> res(n);
        std::stack<int> s;
        // 倒着往栈里放
        for (int i = n - 1; i >= 0; i--) {
            // 删掉 nums[i] 后面较大的元素
            while (!s.empty() && s.top() > nums[i]) {
```

```

        s.pop();
    }
    // 现在栈顶就是 nums[i] 身后的更小或相等元素
    res[i] = s.empty() ? -1 : s.top();
    s.push(nums[i]);
}
return res;
}
};

```

901. 股票价格跨度

设计一个算法收集某些股票的每日报价，并返回该股票当日价格的 **跨度**。

当日股票价格的 **跨度** 被定义为股票价格小于或等于今天价格的最大连续日数（从今天开始往回数，包括今天）。

- 例如，如果未来 7 天股票的价格是 `[100,80,60,70,60,75,85]`，那么股票跨度将是 `[1,1,1,2,1,4,6]`。

实现 `StockSpanner` 类：

- `StockSpanner()` 初始化类对象。
- `int next(int price)` 给出今天的股价 `price`，返回该股票当日价格的 **跨度**。

示例：

输入：

`["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]`

`],[100],[80],[60],[70],[60],[75],[85]`

输出：

`[null, 1, 1, 1, 2, 1, 4, 6]`

解释：

`StockSpanner stockSpanner = new StockSpanner();`

`stockSpanner.next(100);` // 返回 1

`stockSpanner.next(80);` // 返回 1

`stockSpanner.next(60);` // 返回 1

`stockSpanner.next(70);` // 返回 2

`stockSpanner.next(60);` // 返回 1

`stockSpanner.next(75);` // 返回 4，因为截至今天的最后 4 个股价 (包括今天的股价 75) 都小于或等于今天的股价。

`stockSpanner.next(85);` // 返回 6

提示：

- `1 <= price <= 105`

- 最多调用 next 方法 104 次

题目来源：[力扣 901. 股票价格跨度](#)。

这道题显然要用到 [单调栈技巧](#)：当加入 price 时，把所有小于等于 price 的价格都「挤掉」，相当于计算前一个更大元素，即 [单调栈的几种模板实现](#) 中的 prevGreaterElement 函数。

比如已经入栈的价格序列是 [40, 30, 20, 10]，那么如果执行 next(25)，价格序列变成 [40, 30, 25]，20 和 10 都会被「挤掉」，算上 25 本身，函数返回 2 + 1 = 3。

但还有个问题，这个 3 应该作为「权重」和 25 一同存储在栈中。因为之后 25 还可能被挤掉，比如说执行 next(26)，价格序列就变成了 [40, 30, 26]，但这种情况下之前的 20 和 10 显然也应该被挤掉，函数应该返回 3 + 1 = 4。具体解法看代码吧。

```
class StockSpanner {
    // int[] 记录 {价格, 小于等于该价格的天数} 二元组
    stack<pair<int, int>> stk;

public:
    int next(int price) {
        // 算上当天
        int count = 1;
        // 单调栈模板
        while (!stk.empty() && price >= stk.top().first) {
            // 挤掉价格低于 price 的记录
            pair<int, int> prev = stk.top();
            stk.pop();
            // 计算小于等于 price 的天数
            count += prev.second;
        }
        stk.push({price, count});

        return count;
    }
};
```

402. 移掉 K 位数字

给你一个以字符串表示的非负整数 num 和一个整数 k，移除这个数中的 k 位数字，使得剩下的数字最小。请你以字符串形式返回这个最小的数字。

示例 1：

输入： num = "1432219", k = 3

输出： "1219"

解释： 移除掉三个数字 4, 3, 和 2 形成一个新的最小的数字 1219。

示例 2：

输入：num = "10200", k = 1

输出："200"

解释：移掉首位的 1 剩下的数字为 200. 注意输出不能有任何前导零。

示例 3：

输入：num = "10", k = 2

输出："0"

解释：从原数字移除所有的数字，剩余为空就是 0。

提示：

- $1 \leq k \leq \text{num.length} \leq 105$
- num 仅由若干位数字（0 - 9）组成
- 除了 0 本身之外，num 不含任何前导零

题目来源：[力扣 402. 移掉 K 位数字](#)。

如果能让结果尽可能小，那么清除数字分两步：

1、先删除 num 中的若干数字，使得 num 从左到右每一位都单调递增。比如 14329 转化成 129，这需要使用到 [单调栈技巧](#)。

2、num 中的每一位变成单调递增的之后，如果 k 还大于 0（还可以继续删除）的话，则删除尾部的数字，比如 129 删除成 12。

```
#include <string>
#include <stack>
#include <algorithm>

class Solution {
public:
    std::string removeKdigits(std::string num, int k) {
        std::stack<char> stk;
        for (char c : num) {
            // 单调栈代码模板
            while (!stk.empty() && c < stk.top() && k > 0) {
                stk.pop();
                k--;
            }
            // 防止 0 作为数字的开头
            if (stk.empty() && c == '0') {
                continue;
            }
            stk.push(c);
        }
    }
};
```

```

// 此时栈中元素单调递增，若 k 还没用完的话删掉栈顶元素
while (k > 0 && !stk.empty()) {
    stk.pop();
    k--;
}
// 若最后没剩下数字，就是 0
if (stk.empty()) {
    return "0";
}
// 将栈中字符转化成字符串
std::string result = "";
while (!stk.empty()) {
    result += stk.top();
    stk.pop();
}
// 出栈顺序和字符串顺序是反的
std::reverse(result.begin(), result.end());
return result;
}
};

```

853. 车队

在一条单行道上，有 n 辆车开往同一目的地。目的地是几英里以外的 $target$ 。

给定两个整数数组 $position$ 和 $speed$ ，长度都是 n ，其中 $position[i]$ 是第 i 辆车的位置， $speed[i]$ 是第 i 辆车的速度(单位是英里/小时)。

一辆车永远不会超过前面的另一辆车，但它可以追上去，并以较慢车的速度在另一辆车旁边行驶。

车队 是指并排行驶的一辆或几辆汽车。车队的速度是车队中 **最慢** 的车的速度。

即便一辆车在 $target$ 才赶上了一个车队，它们仍然会被视作是同一个车队。

返回到达目的地的车队数量。

示例 1：

输入： $target = 12$, $position = [10,8,0,5,3]$, $speed = [2,4,1,1,3]$

输出： 3

解释：

- 从 10（速度为 2）和 8（速度为 4）开始的车会组成一个车队，它们在 12 相遇。车队在 $target$ 形成。
- 从 0（速度为 1）开始的车不会追上其它任何车，所以它自己是一个车队。

- 从 5（速度为 1）和 3（速度为 3）开始的车组成一个车队，在 6 相遇。车队以速度 1 移动直到它到达 target。

示例 2:

输入: target = 10, position = [3], speed = [3]

输出: 1

解释:

只有一辆车，因此只有一个车队。

示例 3:

输入: target = 100, position = [0,2,4], speed = [4,2,1]

输出: 1

解释:

- 从 0（速度为 4）和 2（速度为 2）开始的车组成一个车队，在 4 相遇。从 4 开始的车（速度为 1）移动到了 5。
- 然后，在 4（速度为 2）的车队和 5（速度为 1）的车成为一个车队，在 6 相遇。车队以速度 1 移动直到它到达 target。

提示:

- `n == position.length == speed.length`
- `1 <= n <= 105`
- `0 < target <= 106`
- `0 <= position[i] < target`
- position 中每个值都 **不同**
- `0 < speed[i] <= 106`

题目来源: [力扣 853. 车队](#)。

这题考察「单调栈」结构的使用。是否能够形成车队，取决于下述规律：

如果车 x 排在车 y 后面，且 x 到达终点所需时间比 y 少，则 x 必然会被 y 卡住，形成车队。

所以本题的思路是先根据每辆车的起始位置 position 排序，然后计算出时间数组 time。

假设计算出的 time 数组为 [12, 3, 7, 1, 2]，那么观察数组的单调性变化，最后肯定会形成三个车队，他们到达终点的时间分别是 12, 7, 2。

可以利用单调栈结构模拟得出结果，不过效率稍微低一些。也可以倒序遍历数组得出递增子序列，子序列的长度即答案。


```

#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {
        int n = position.size();
        vector<vector<int>> cars(n, vector<int>(2));
        for (int i = 0; i < n; i++) {
            cars[i][0] = position[i];
            cars[i][1] = speed[i];
        }
        // 按照初始位置，从小到大排序
        sort(cars.begin(), cars.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[0] < b[0];
        });

        // 计算每辆车到达终点的时间
        vector<double> time(n);
        for (int i = 0; i < n; i++) {
            int pos = cars[i][0];
            int vel = cars[i][1];
            time[i] = (double)(target - pos) / vel;
        }

        // 使用单调栈计算车队的数量
        // 避免使用栈模拟，倒序遍历取递增序列就是答案
        int res = 0;
        double maxTime = 0; // 这里maxTime相当于栈顶
        for (int i = n - 1; i >= 0; i--) {
            if (time[i] > maxTime) {
                maxTime = time[i];
                res++;
            }
        }
        return res;
    }
};

```

581.最短无序连续子数组

给你一个整数数组 `nums` ，你需要找出一个 **连续子数组** ，如果对这个子数组进行升序排序，那么整个数组都会变为升序排序。

请你找出符合题意的 **最短** 子数组，并输出它的长度。

示例 1：

输入: nums = [2,6,4,8,10,9,15]

输出: 5

解释: 你只需要对 [6, 4, 8, 10, 9] 进行升序排序, 那么整个表都会变为升序排序。

示例 2:

输入: nums = [1,2,3,4]

输出: 0

示例 3:

输入: nums = [1]

输出: 0

提示:

- `1 <= nums.length <= 104`
- `-105 <= nums[i] <= 105`

进阶: 你可以设计一个时间复杂度为 $O(n)$ 的解决方案吗?

题目来源: [力扣 581. 最短无序连续子数组](#)。

最简单的解法是排序, 排序之后很容易看出来哪一部分子数组乱序了。这里主要介绍一下单调栈的解法。

单调递增栈会筛选出递增的元素序列, 换句话说, 每加入一个新元素 `x`, 就会弹出栈顶大于 `x` 的其他元素, 直到栈顶元素小于 `x` 为止。

反过来, 单调递减栈会筛选出递减的元素序列, 换句话说, 每加入一个新元素 `x`, 就会弹出栈顶小于 `x` 的其他元素, 直到栈顶元素大于 `x` 为止。

综上, 如果正序遍历 `nums`, 维护一个递增栈, 那么弹出的元素就是乱序的元素; 如果反向遍历 `nums`, 维护一个递减栈, 那么弹出的元素就是乱序的元素。

```
// 排序解法 (双指针)
class Solution {
public:
    int findUnsortedSubarray(vector<int>& nums) {
        vector<int> temp = nums;
        sort(temp.begin(), temp.end());
        int left = INT_MAX, right = INT_MIN;
        for (int i = 0; i < nums.size(); i++) {
            if (temp[i] != nums[i]) {
                left = i;
                break;
            }
        }
        for (int i = nums.size() - 1; i >= 0; i--) {
```

```

        if (temp[i] != nums[i]) {
            right = i;
            break;
        }
    }
    if (left == INT_MAX && right == INT_MIN) {
        // nums 本来就是有序的
        return 0;
    }
    return right - left + 1;
}
};

// 单调栈解法（个人感觉完全没必要这么做）
class Solution2 {
public:
    int findUnsortedSubarray(vector<int>& nums) {
        int n = nums.size();
        int left = INT_MAX, right = INT_MIN;
        // 递增栈，存储元素索引
        stack<int> incrStk;
        for (int i = 0; i < n; i++) {
            while (!incrStk.empty() && nums[incrStk.top()] > nums[i]) {
                // 弹出的元素都是乱序元素，其中最小的索引就是乱序子数组的左边界
                left = min(left, incrStk.top());
                incrStk.pop();
            }
            incrStk.push(i);
        }
        // 递减栈，存储元素索引
        stack<int> decrStk;
        for (int i = n - 1; i >= 0; i--) {
            while (!decrStk.empty() && nums[decrStk.top()] < nums[i]) {
                // 弹出的元素都是乱序元素，其中最大的索引就是乱序子数组的右边界
                right = max(right, decrStk.top());
                decrStk.pop();
            }
            decrStk.push(i);
        }
        if (left == INT_MAX && right == INT_MIN) {
            // 说明单调栈没有弹出任何元素，即 nums 本来就是有序的
            return 0;
        }
        return right - left + 1;
    }
};

```

单调队列

单调队列结构解决滑动窗口问题

前文用 [单调栈解决三道算法问题](#) 介绍了单调栈这种特殊数据结构，本文写一个类似的数据结构「单调队列」。

也许这种数据结构的名字你没听过，其实没啥难的，就是一个「队列」，只是使用了一点巧妙的方法，使得队列中的元素全都是单调递增（或递减）的。

为啥要发明「单调队列」这种结构呢，主要是为了解决下面这个场景：

给你一个数组 `window`，已知其最值为 `A`，如果给 `window` 中添加一个数 `B`，那么比较一下 `A` 和 `B` 就可以立即算出新的最值；但如果要从 `window` 数组中减少一个数，就不能直接得到最值了，因为如果减少的这个数恰好是 `A`，就需要遍历 `window` 中的所有元素重新寻找新的最值。

这个场景很常见，但不用单调队列似乎也可以，比如 [优先级队列（二叉堆）](#) 就是专门用来动态寻找最值的，我创建一个大（小）顶堆，不就可以很快拿到最大（小）值了吗？

如果单纯地维护最值的话，优先级队列很专业，队头元素就是最值。但优先级队列无法满足标准队列结构「先进先出」的**时间顺序**，因为优先级队列底层利用二叉堆对元素进行动态排序，元素的出队顺序是元素的大小顺序，和入队的先后顺序完全没有关系。

所以，现在需要一种新的队列结构，既能够维护队列元素「先进先出」的时间顺序，又能够正确维护队列中所有元素的最值，这就是「单调队列」结构。

「单调队列」这个数据结构主要用来辅助解决滑动窗口相关的问题，前文 [滑动窗口核心框架](#) 把滑动窗口算法作为双指针技巧的一部分进行了讲解，但有些稍微复杂的滑动窗口问题不能只靠两个指针来解决，需要上更先进的数据结构。

比方说，你注意看前文 [滑动窗口核心框架](#) 讲的几道题目，每当窗口扩大（`right++`）和窗口缩小（`left++`）时，你单凭移出和移入窗口的元素即可决定是否更新答案。

但本文开头说的那个判断一个窗口中最值的例子，你无法单凭移出窗口的那个元素更新窗口的最值，除非重新遍历所有元素，但这样的话时间复杂度就上来了，这是我们不希望看到的。

我们来看看力扣第 239 题「[滑动窗口最大值](#)」，就是一道标准的滑动窗口问题：

给你输入一个数组 `nums` 和一个正整数 `k`，有一个大小为 `k` 的窗口在 `nums` 上从左至右滑动，请你输出每次窗口中 `k` 个元素的最大值。

函数签名如下：

```
vector<int> maxSlidingWindow(vector<int>& nums, int k);
```

比如说力扣给出的一个示例：

```
输入: nums = [1,3,-1,-3,5,3,6,7], k = 3
输出: [3,3,5,5,6,7]
```

解释：

滑动窗口的位置	最大值
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

接下来，我们就借助单调队列结构，用 $O(1)$ 时间算出每个滑动窗口中的最大值，使得整个算法在线性时间完成。

在介绍「单调队列」这种数据结构的 API 之前，先来对比一下 [普通的队列](#) 的标准 API 和单调队列实现的 API：

```
// 普通队列的 API
class Queue {
public:
    // enqueue 操作，在队尾加入元素 n
    void push(int n);
    // dequeue 操作，删除队头元素
    void pop();
};

// 单调队列的 API
class MonotonicQueue {
public:
    // 在队尾添加元素 n
    void push(int n);
    // 返回当前队列中的最大值
    int max();
    // 队头元素如果是 n，删除它
    void pop(int n);
};
```

当然，单调队列这几个 API 的实现方法肯定跟一般的 Queue 不一样，不过我们暂且不管，而且认为这几个操作的时间复杂度都是 $O(1)$ ，先把这道「滑动窗口」问题的解答框架搭出来：

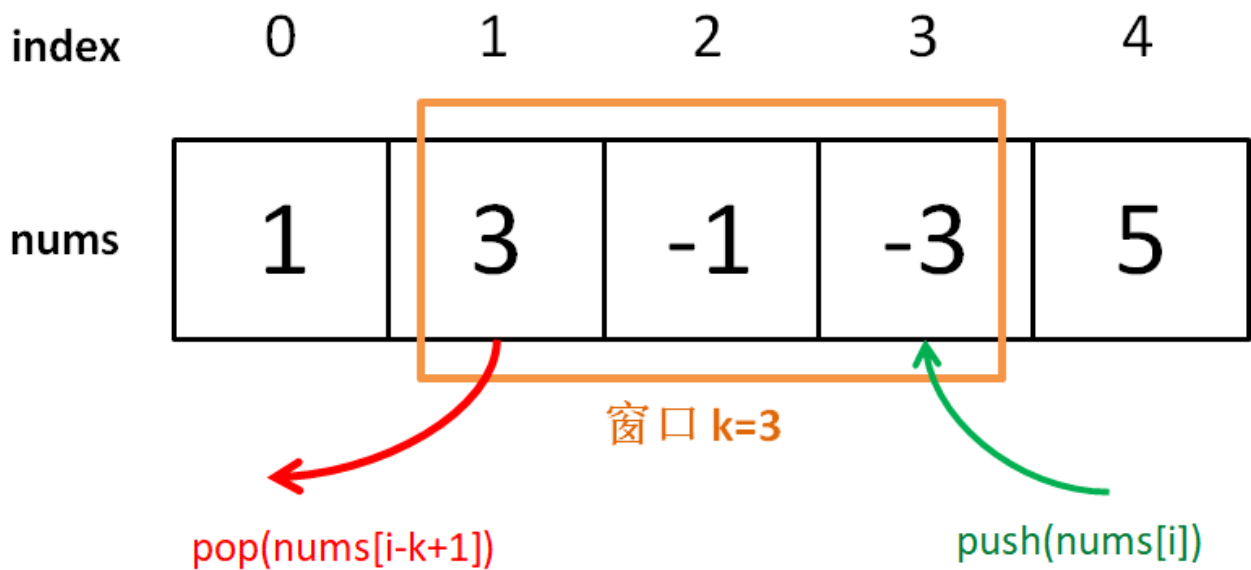
```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    MonotonicQueue window;
    vector<int> res;

    for (int i = 0; i < nums.size(); i++) {
        if (i < k - 1) {
            // 先把窗口的前 k - 1 填满
            window.push(nums[i]);
        } else {
```

```

        // 窗口开始向前滑动
        // 移入新元素
        window.push(nums[i]);
        // 将当前窗口中的最大元素记入结果
        res.push_back(window.max());
        // 移出最后的元素
        window.pop(nums[i - k + 1]);
    }
}
return res;
}

```



这个思路很简单吧，下面我们开始重头戏，单调队列的实现。

观察滑动窗口的过程就能发现，实现「单调队列」必须使用一种数据结构支持在头部和尾部进行插入和删除，很明显 [双链表](#) 是满足这个条件的。

「单调队列」的核心思路和「单调栈」类似，`push` 方法依然在队尾添加元素，但是要把前面比自己小的元素都删掉：

```

#include <list>
using namespace std;

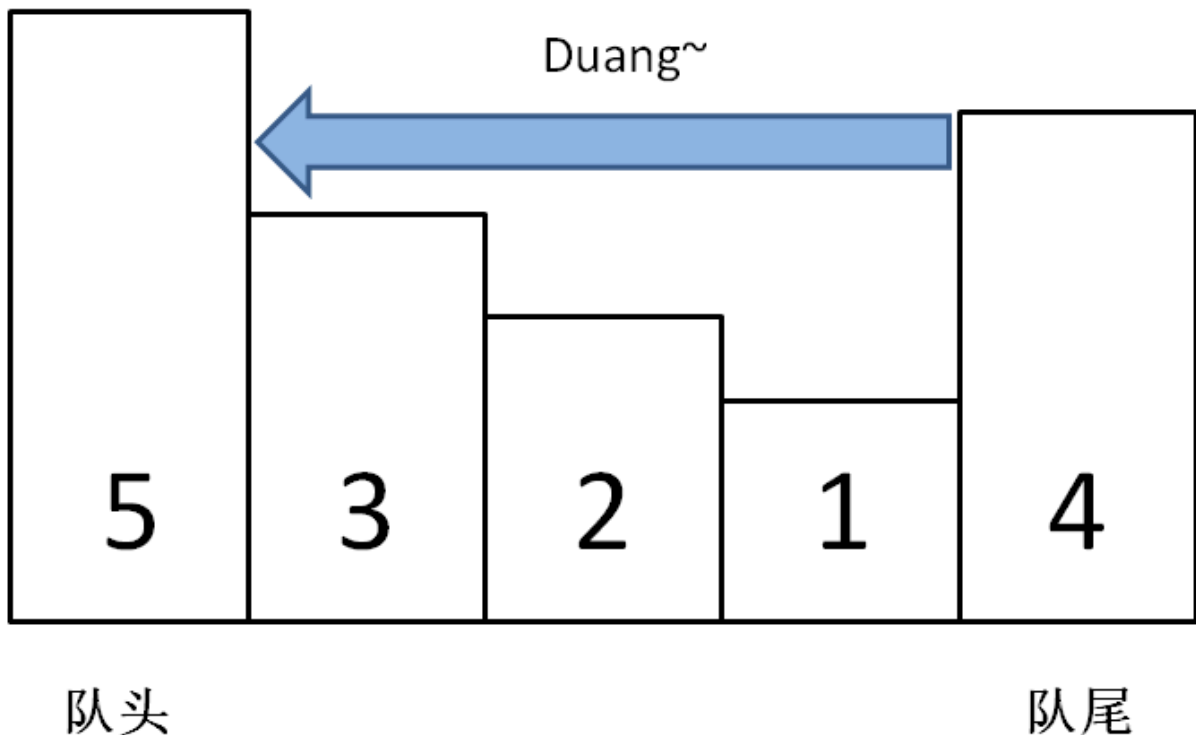
class MonotonicQueue {
    // 双链表，支持快速在头部和尾部增删元素
    // 维护其中的元素自尾部到头部单调递增
    list<int> maxq;

public:
    // 在尾部添加一个元素 n，维护 maxq 的单调性质
    void push(int n) {
        // 将前面小于自己的元素都删除
    }
}

```

```
while (!maxq.empty() && maxq.back() < n) {  
    maxq.pop_back();  
}  
maxq.push_back(n);  
}  
  
};
```

你可以想象，加入数字的大小代表人的体重，体重大的会把前面体重不足的压扁，直到遇到更大的量级才停住。

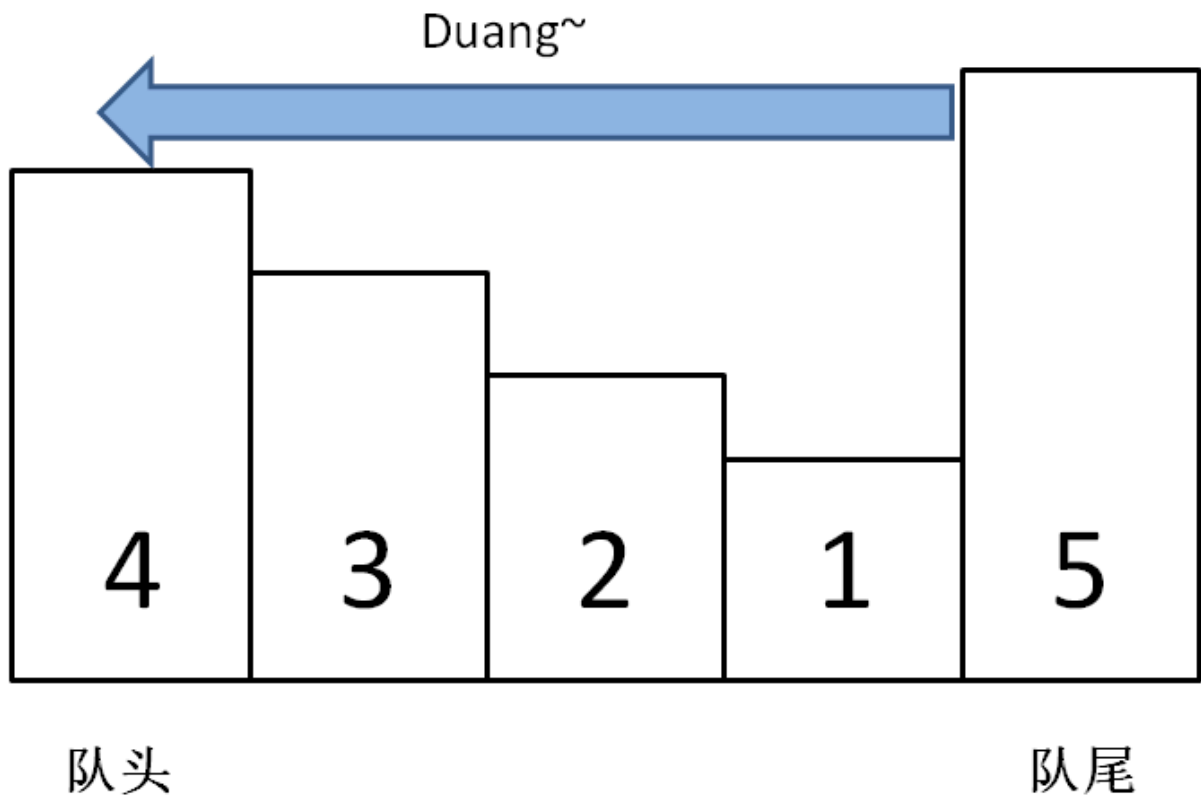


如果每个元素被加入时都这样操作，最终单调队列中的元素大小就会保持一个**单调递减**的顺序，因此我们的 `max` 方法就很好写了，只要把队头元素返回即可；`pop` 方法也是操作队头，如果队头元素是待删除元素 `n`，那么就删除它：

```
class MonotonicQueue {  
    // 为了节约篇幅，省略上文给出的代码部分...  
  
public:  
    int max() {  
        // 队头的元素肯定是最大的  
        return maxq.front();  
    }  
  
    void pop(int n) {  
        if (n == maxq.front()) {  
            maxq.pop_front();  
        }  
    }  
};
```

```
    }  
    }  
};
```

pop 方法之所以要判断 `n == maxq.getFirst()`，是因为我们想删除的队头元素 `n` 可能已经在 push 的过程中被「压扁」了，可能已经不存在了，这种情况就不用删除了：



至此，单调队列设计完毕，看下完整的解题代码：

```
// 单调队列的实现  
class MonotonicQueue {  
    deque<int> maxq;  
public:  
    void push(int n) {  
        // 将小于 n 的元素全部删除  
        while (!maxq.empty() && maxq.back() < n) {  
            maxq.pop_back();  
        }  
        // 然后将 n 加入尾部  
        maxq.push_back(n);  
    }  
  
    int max() {  
        return maxq.front();  
    }  
  
    void pop(int n) {
```



```

        if (n == maxq.front()) {
            maxq.pop_front();
        }
    }
};

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        MonotonicQueue window;
        vector<int> res;

        for (int i = 0; i < nums.size(); i++) {
            if (i < k - 1) {
                // 先填满窗口的前 k - 1
                window.push(nums[i]);
            } else {
                // 窗口向前滑动，加入新数字
                window.push(nums[i]);
                // 记录当前窗口的最大值
                res.push_back(window.max());
                // 移出旧数字
                window.pop(nums[i - k + 1]);
            }
        }
        return res;
    }
};

```

有一点细节问题不要忽略，在实现 `MonotonicQueue` 时，我们使用了 Java 的 `LinkedList`，因为链表结构支持在头部和尾部快速增删元素；而在解法代码中的 `res` 则使用的 `ArrayList` 结构，因为后续会按照索引取元素，所以数组结构更合适。其他语言的实现也要注意这些细节。

关于单调队列 API 的时间复杂度，读者可能有疑惑：`push` 操作中含有 `while` 循环，最坏情况下的时间复杂度应该 $O(N)$ 呀，再加上一层 `for` 循环，本算法的时间复杂度应该是 $O(N^2)$ 才对吧？

这里就用到了 [算法时空复杂度分析指南](#) 中讲到的摊还分析：

单独看 `push` 操作，最坏时间复杂度确实是 $O(N)$ ，但是平均时间复杂度是 $O(1)$ 。我们一般用平均复杂度而不是最坏时间复杂度来衡量 API 接口，所以这个算法整体的时间复杂度是 $O(N)$ ，而不是 $O(N^2)$ 。

也可以这样从整体上分析：整个算法做的事情就是把 `nums` 中的每个元素加入和移出 `window` **至多一次**，不可能把同一个元素多次移入移出 `window`，所以整体的时间复杂度是 $O(N)$ 。

空间复杂度很容易分析，就是窗口的大小 $O(k)$ 。

最后，我提出几个问题请大家思考：

- 1、本文给出的 `MonotonicQueue` 类只实现了 `max` 方法，你是否能够再额外添加一个 `min` 方法，在 $O(1)$ 的时间返回队列中所有元素的最小值？
- 2、本文给出的 `MonotonicQueue` 类的 `pop` 方法还需要接收一个参数，这不那么优雅，而且有悖于标准队列的 API，请你修复这个缺陷。
- 3、请你实现 `MonotonicQueue` 类的 `size` 方法，返回单调队列中元素的个数（注意，由于每次 `push` 方法都可能从底层的 `q` 列表中删除元素，所以 `q` 中的元素个数并不是单调队列的元素个数）。

也就是说，你是否能够实现单调队列的通用实现：

```
// 单调队列的通用实现，可以高效维护最大值和最小值
template <typename E>
class MonotonicQueue {
public:
    // 标准队列 API，向队尾加入元素
    void push(E elem);

    // 标准队列 API，从队头弹出元素，符合先进先出的顺序
    E pop();

    // 标准队列 API，返回队列中的元素个数
    int size();

    // 单调队列特有 API， $O(1)$  时间计算队列中元素的最大值
    E max();

    // 单调队列特有 API， $O(1)$  时间计算队列中元素的最小值
    E min();
};
```

通用实现

我们原先的简陋实现包含了 `max` 方法的实现，其原理是在底层维护了一个队列 `maxq`，维护这个队列中从尾部到头部的元素单调递增。

那么实现 `min` 方法也是类似的，可以在底层再维护一个 `minq` 队列，维护队列中元素从尾部到头部的元素单调递减，这样头部第一个元素就是所有元素中的最小值了。

当然，由于 `push` 方法在添加元素的同时还可能会删除元素，所以 `maxq` 和 `minq` 中都没有保存所有元素。如果想实现标准的 `pop` 方法以及 `size` 方法，我们还得再额外维护一个标准队列 `q`，这个 `q` 存储所有存在于队列中的元素，不维护单调性。

```
#include <deque>
#include <queue>
```

```

#include <stdexcept>

template <typename E>
class MonotonicQueue {
private:
    std::queue<E> q;           // 标准队列，用于维护逻辑顺序和 size
    std::deque<E> max_dq;     // 单调递减队列，维护最大值
    std::deque<E> min_dq;     // 单调递增队列，维护最小值

public:
    // 标准队列 API：向队尾加入元素
    void push(E elem) {
        // 1. 维护标准队列
        q.push(elem);

        // 2. 维护 max_dq (递减)
        // 将小于 elem 的元素全部删除
        while (!max_dq.empty() && max_dq.back() < elem) {
            max_dq.pop_back();
        }
        max_dq.push_back(elem);

        // 3. 维护 min_dq (递增)
        // 将大于 elem 的元素全部删除
        while (!min_dq.empty() && min_dq.back() > elem) {
            min_dq.pop_back();
        }
        min_dq.push_back(elem);
    }

    // 标准队列 API：从队头弹出元素
    E pop() {
        if (q.empty()) {
            throw std::runtime_error("Queue is empty");
        }

        E front_val = q.front();
        q.pop();

        // 如果弹出的元素正好是当前的最大值或最小值，则同步弹出单调队列首部
        if (front_val == max_dq.front()) {
            max_dq.pop_front();
        }
        if (front_val == min_dq.front()) {
            min_dq.pop_front();
        }

        return front_val;
    }
}

```

```

// 标准队列 API: 返回队列中的逻辑元素个数
int size() {
    return q.size();
}

// 单调队列特有 API: O(1) 计算最大值
E max() {
    if (max_dq.empty()) throw std::runtime_error("Queue is empty");
    return max_dq.front();
}

// 单调队列特有 API: O(1) 计算最小值
E min() {
    if (min_dq.empty()) throw std::runtime_error("Queue is empty");
    return min_dq.front();
}

bool empty() {
    return q.empty();
}
};

```

1. 为什么 pop 不需要参数了？

在之前的实现中，`pop(n)` 需要调用者手动传入“本该出队的值”来判断是否需要更新单调队列。现在我们引入了 `std::queue q`，它完整保存了所有进入队列的元素顺序。当我们调用 `pop()` 时，`q.front()` 明确告诉了我们当前逻辑上出队的元素是谁，我们只需拿这个值去和 `max_dq / min_dq` 的队头比对即可。

2. 复杂度分析

• 时间复杂度：

- `push`：每个元素最多入队一次、出队一次，均摊复杂度为 $O(1)$ 。
- `pop`： $O(1)$ 。
- `max / min / size`： $O(1)$ 。

- **空间复杂度**： $O(N)$ ，其中 N 是当前队列中的元素个数（需要三个底层容器存储）。

3. 处理重复元素

注意在 `push` 的 `while` 循环中，我们使用的是 `max_dq.back() < elem` 而不是 `<=`。

原理：如果队列中有两个相同的最大值（例如 `[5, 3, 5]`），`max_dq` 会存储两个 5。当第一个 5 出队时，`max_dq` 弹出一个；第二个 5 依然能保证 `max()` 返回正确的结果。

经典习题

单调队列 + 滑动窗口

单调队列可以和 [滑动窗口](#) 算法结合，在窗口滑动的过程中快速计算窗口内部的最值。

1438. 绝对差不超过限制的最长连续子数组

给你一个整数数组 `nums`，和一个表示限制的整数 `limit`，请你返回最长连续子数组的长度，该子数组中的任意两个元素之间的绝对差必须小于或者等于 `limit`。

如果不存在满足条件的子数组，则返回 `0`。

示例 1:

输入: `nums = [8,2,4,7], limit = 4`

输出: `2`

解释: 所有子数组如下:

`[8]` 最大绝对差 $|8-8| = 0 \leq 4$.

`[8,2]` 最大绝对差 $|8-2| = 6 > 4$.

`[8,2,4]` 最大绝对差 $|8-2| = 6 > 4$.

`[8,2,4,7]` 最大绝对差 $|8-2| = 6 > 4$.

`[2]` 最大绝对差 $|2-2| = 0 \leq 4$.

`[2,4]` 最大绝对差 $|2-4| = 2 \leq 4$.

`[2,4,7]` 最大绝对差 $|2-7| = 5 > 4$.

`[4]` 最大绝对差 $|4-4| = 0 \leq 4$.

`[4,7]` 最大绝对差 $|4-7| = 3 \leq 4$.

`[7]` 最大绝对差 $|7-7| = 0 \leq 4$.

因此，满足题意的最长子数组的长度为 `2`。

示例 2:

输入: `nums = [10,1,2,4,7,2], limit = 5`

输出: `4`

解释: 满足题意的最长子数组是 `[2,4,7,2]`，其最大绝对差 $|2-7| = 5 \leq 5$ 。

示例 3:

输入: `nums = [4,2,2,2,4,4,2,2], limit = 0`

输出: `3`

提示:

- `1 \leq nums.length \leq 10^5`
- `1 \leq nums[i] \leq 10^9`
- `0 \leq limit \leq 10^9`

题目来源: [力扣 1438. 绝对差不超过限制的最长连续子数组](#)。

当窗口内绝对值之差不超过 `limit` 时扩大窗口，当新加入窗口的元素使得绝对值之差超过 `limit` 时开始收缩窗口，窗口的最大宽度即最长子数组的长度。

但有个问题，当窗口进新元素时，我可以更新窗口中的最大值和最小值，但当窗口收缩时，如何更新最大值和最小值呢？难道要遍历一遍窗口中的所有元素吗？这就用到单调队列结构了，

这里需要一个通用的 `MonotonicQueue` 类，用来高效判断窗口中的最大值和最小值。

```
#include <deque>
#include <vector>
#include <cassert>
using namespace std;

// 下面给出的是单调队列的通用实现，运行速度可能较慢，你可以自行简化提升速度
// 单调队列的详细解析见
// https://labuladong.online/algorithm/problem-set/monotonic-queue/
class MonotonicQueue {
    // 常规队列，存储所有元素
    deque<int> q;
    // 元素降序排列的单调队列，头部是最大值
    deque<int> maxq;
    // 元素升序排列的单调队列，头部是最小值
    deque<int> minq;

public:
    void push(int elem) {
        // 维护常规队列，直接在队尾插入元素
        q.push_back(elem);

        // 维护 maxq，将小于 elem 的元素全部删除
        while (!maxq.empty() && maxq.back() < elem) {
            maxq.pop_back();
        }
        maxq.push_back(elem);

        // 维护 minq，将大于 elem 的元素全部删除
        while (!minq.empty() && minq.back() > elem) {
            minq.pop_back();
        }
        minq.push_back(elem);
    }

    int max() {
        // maxq 的头部是最大元素
        return maxq.front();
    }

    int min() {
        // minq 的头部是最大元素
        return minq.front();
    }

    int pop() {
        // 从标准队列头部弹出需要删除的元素
        int deleteVal = q.front();
```

```

        q.pop_front();

        // 由于 push 的时候会删除元素，deleteVal 可能已经被删掉了
        if (deleteVal == maxq.front()) {
            maxq.pop_front();
        }
        if (deleteVal == minq.front()) {
            minq.pop_front();
        }
        return deleteVal;
    }

    int size() {
        // 标准队列的大小即是当前队列的大小
        return q.size();
    }

    bool isEmpty() {
        return q.empty();
    }
};

class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        int left = 0, right = 0;
        int windowSize = 0, res = 0;
        MonotonicQueue window;
        // 滑动窗口模板
        while (right < nums.size()) {
            // 扩大窗口，更新窗口最值
            window.push(nums[right]);
            right++;
            windowSize++;
            while (window.max() - window.min() > limit) {
                // 缩小窗口，更新窗口最值
                window.pop();
                left++;
                windowSize--;
            }
            // 在窗口收缩判断完之后才更新答案
            res = max(res, windowSize);
        }
        return res;
    }
};

```

862. 和至少为 K 的最短子数组

给你一个整数数组 `nums` 和一个整数 `k`，找出 `nums` 中和至少为 `k` 的 **最短非空子数组**，并返回该子数组的长度。如果不存在这样的 **子数组**，返回 `-1`。

子数组 是数组中 **连续** 的一部分。

示例 1:

输入: `nums = [1]`, `k = 1`

输出: `1`

示例 2:

输入: `nums = [1,2]`, `k = 4`

输出: `-1`

示例 3:

输入: `nums = [2,-1,2]`, `k = 3`

输出: `3`

提示:

- `1 <= nums.length <= 105`
- `-105 <= nums[i] <= 105`
- `1 <= k <= 109`

题目来源: [力扣 862. 和至少为 K 的最短子数组](#)。

这题的难度是比较大的，难点在于同时结合了 [滑动窗口算法](#)、[前缀和技巧](#) 和 [单调队列](#) 几个知识点，建议你先理解这三篇文章的要义，否则看不懂这题的解法。

首先，想要快速记录子数组的和，需要 [前缀和技巧](#) 预计算一个 `preSum` 数组，然后在这个 `preSum` 数组上施展 [滑动窗口算法](#) 寻找一个差值大于等于 `k` 且宽度最小的「窗口」，这个窗口的大小就是题目想要的结果。

这里面还有个问题，当滑动窗口扩大时，新进入窗口的元素 `preSum[right]` 需要知道窗口中最小的那个元素是多少，和最小的那个元素相减才能得到尽可能大的子数组和。

如何快速判断窗口中的最值？这就需要单调队列结构出马了，直接看解法代码吧。

注意，此题使用单调队列是因为数组为整数数组，可能有负数，`presum`数组不是逐个增加的，所以不能用窗口右端－左端

```
#include <vector>
#include <deque>
#include <limits.h>
```

```
// 下面给出的是单调队列的通用实现，运行速度可能较慢，你可以自行简化提升速度
// 单调队列的详细解析见
```



```
// https://labuladong.online/algo/problem-set/monotonic-queue/
class MonotonicQueue {
public:
    // 常规队列，存储所有元素
    std::deque<long> q;
    // 元素降序排列的单调队列，头部是最大值
    std::deque<long> maxq;
    // 元素升序排列的单调队列，头部是最小值
    std::deque<long> minq;

    void push(long elem) {
        // 维护常规队列，直接在队尾插入元素
        q.push_back(elem);

        // 维护 maxq，将小于 elem 的元素全部删除
        while (!maxq.empty() && maxq.back() < elem) {
            maxq.pop_back();
        }
        maxq.push_back(elem);

        // 维护 minq，将大于 elem 的元素全部删除
        while (!minq.empty() && minq.back() > elem) {
            minq.pop_back();
        }
        minq.push_back(elem);
    }

    long max() {
        // maxq 的头部是最大元素
        return maxq.front();
    }

    long min() {
        // minq 的头部是最大元素
        return minq.front();
    }

    long pop() {
        // 从标准队列头部弹出需要删除的元素
        long deleteVal = q.front();
        q.pop_front();

        // 由于 push 的时候会删除元素，deleteVal 可能已经被删掉了
        if (deleteVal == maxq.front()) {
            maxq.pop_front();
        }
        if (deleteVal == minq.front()) {
            minq.pop_front();
        }
        return deleteVal;
    }
};
```

```

    }

    int size() {
        // 标准队列的大小即是当前队列的大小
        return q.size();
    }

    bool isEmpty() {
        return q.empty();
    }
};

class Solution {
public:
    int shortestSubarray(std::vector<int>& nums, int k) {
        int n = nums.size();
        // 看题目的数据范围，前缀和数组中元素可能非常大，所以用 long 类型
        std::vector<long> preSum(n + 1, 0);
        // 计算 nums 的前缀和数组
        for (int i = 1; i <= n; ++i) {
            preSum[i] = preSum[i - 1] + nums[i - 1];
        }
        // 单调队列结构辅助滑动窗口算法
        MonotonicQueue window;
        int right = 0, left = 0;
        int len = INT_MAX;
        // 开始执行滑动窗口算法框架
        while (right < preSum.size()) {
            // 扩大窗口，元素入队
            window.push(preSum[right]);
            ++right;
            // 若新进入窗口的元素和窗口中的最小值之差大于等于 k，
            // 说明得到了符合条件的子数组，缩小窗口，使子数组长度尽可能小
            while (right < preSum.size() && !window.isEmpty()
                && preSum[right] - window.min() >= k) {
                // 更新答案
                len = std::min(len, right - left);
                // 缩小窗口
                window.pop();
                ++left;
            }
        }
        return len == INT_MAX ? -1 : len;
    }
};

```

单调队列 + 环形数组

单调队列还可以在环形数组的场景下排上用场。之前 [单调栈原理](#) 中讲了只要把数组的长度 n 加倍就可以模拟环形数组，但如果让你在环形数组中计算子数组的元素和就需要单调队列辅助了，因为你要保证子数组的长度不能超过 n 。

918. 环形子数组的最大和

给定一个长度为 n 的环形整数数组 `nums`，返回 `nums` 的非空子数组的最大可能和。

环形数组 意味着数组的末端将会与开头相连呈环状。形式上，`nums[i]` 的下一个元素是 `nums[(i + 1) % n]`，`nums[i]` 的前一个元素是 `nums[(i - 1 + n) % n]`。

子数组 最多只能包含固定缓冲区 `nums` 中的每个元素一次。形式上，对于子数组 `nums[i]`，`nums[i + 1]`， $\dots$ ，`nums[j]`，不存在 $i \leq k_1$ ， $k_2 \leq j$ 其中 $k_1 \% n == k_2 \% n$ 。

示例 1：

输入：`nums = [1,-2,3,-2]`

输出：3

解释：从子数组 `[3]` 得到最大和 3

示例 2：

输入：`nums = [5,-3,5]`

输出：10

解释：从子数组 `[5,5]` 得到最大和 $5 + 5 = 10$

示例 3：

输入：`nums = [3,-2,2,-3]`

输出：3

解释：从子数组 `[3]` 和 `[3,-2,2]` 都可以得到最大和 3

提示：

- `n == nums.length`
- `1 <= n <= 3 * 104`
- `-3 * 104 <= nums[i] <= 3 * 104`

题目来源：[力扣 918. 环形子数组的最大和](#)。

诚然，这道题有很巧妙的方法，你可以搜索一下「Kadane 算法」来解决这道题。不过为了举一反三地运用我之前讲过的算法技巧，我就结合 [单调队列结构](#) 和前文 [前缀和技巧](#) 写一个更通用的解法。

首先，这道题和 [53. 最大子序和](#) 非常类似，区别在于本题的数组是环形的，所以上一题的动态规划思路和前缀和思路都不能直接套用过来。不过前文 [单调栈结构详解](#) 中讲过处理环形数组的方法，其实就是把原数组大小扩大一倍，这样就能模拟出环形的效果了。

那么本题也可以把 `nums` 数组扩大一倍，计算前缀和数组 `preSum`，借助一个定长为 `nums.length` 的单调队列来计算环形数组中的最大子数组和。具体实现直接看代码吧。

```
#include <vector>
#include <deque>
#include <climits>

using namespace std;

// 下面给出的是单调队列的通用实现，运行速度可能较慢，你可以自行简化提升速度
// 单调队列的详细解析见
// https://labuladong.online/algorithm/problem-set/monotonic-queue/
template<typename E>
class MonotonicQueue {
    // 常规队列，存储所有元素
    deque<E> q;
    // 元素降序排列的单调队列，头部是最大值
    deque<E> maxq;
    // 元素升序排列的单调队列，头部是最小值
    deque<E> minq;

public:
    void push(E elem) {
        // 维护常规队列，直接在队尾插入元素
        q.push_back(elem);

        // 维护 maxq，将小于 elem 的元素全部删除
        while (!maxq.empty() && maxq.back() < elem) {
            maxq.pop_back();
        }
        maxq.push_back(elem);

        // 维护 minq，将大于 elem 的元素全部删除
        while (!minq.empty() && minq.back() > elem) {
            minq.pop_back();
        }
        minq.push_back(elem);
    }

    E max() {
        // maxq 的头部是最大元素
        return maxq.front();
    }

    E min() {
        // minq 的头部是最大元素
        return minq.front();
    }
}
```

```

E pop() {
    // 从标准队列头部弹出需要删除的元素
    E deleteVal = q.front();
    q.pop_front();

    // 由于 push 的时候会删除元素, deleteVal 可能已经被删掉了
    if (deleteVal == maxq.front()) {
        maxq.pop_front();
    }
    if (deleteVal == minq.front()) {
        minq.pop_front();
    }
    return deleteVal;
}

int size() {
    // 标准队列的大小即是当前队列的大小
    return q.size();
}

bool isEmpty() {
    return q.empty();
}
};

class Solution {
public:
    int maxSubarraySumCircular(vector<int>& nums) {
        int n = nums.size();
        // 模拟环状的 nums 数组
        vector<int> preSum(2 * n + 1, 0);
        preSum[0] = 0;
        // 计算环状 nums 的前缀和
        for (int i = 1; i < preSum.size(); i++) {
            preSum[i] = preSum[i - 1] + nums[(i - 1) % n];
        }
        // 记录答案
        int maxSum = INT_MIN;
        // 维护一个滑动窗口, 以便根据窗口中的最小值计算最大子数组和
        MonotonicQueue<int> window;
        window.push(0);
        for (int i = 1; i < preSum.size(); i++) {
            maxSum = max(maxSum, preSum[i] - window.min());
            // 维护窗口的大小为 nums 数组的大小
            if (window.size() == n) {
                window.pop();
            }
            window.push(preSum[i]);
        }
    }
};

```

```
        return maxSum;
    }
};
```

单调队列 + 动态规划

动态规划很多时候会用到嵌套 for 循环计算最值，如果是计算一个窗口中的最值，可以利用单调队列结构维护最值，从而消除一层 for 循环，降低时间复杂度。

1696. 跳跃游戏 VI

给你一个下标从 0 开始的整数数组 `nums` 和一个整数 `k`。

一开始你在下标 0 处。每一步，你最多可以往前跳 `k` 步，但你不能跳出数组的边界。也就是说，你可以从下标 `i` 跳到 `[i + 1, min(n - 1, i + k)]` 包含两个端点的任意位置。

你的目标是到达数组最后一个位置（下标为 `n - 1`），你的 **得分** 为经过的所有数字之和。

请你返回你能得到的 **最大得分**。

示例 1：

输入： `nums = [1,-1,-2,4,-7,3]`, `k = 2`

输出： 7

解释： 你可以选择子序列 `[1,-1,4,3]`（上面加粗的数字），和为 7。

示例 2：

输入： `nums = [10,-5,-2,4,0,3]`, `k = 3`

输出： 17

解释： 你可以选择子序列 `[10,4,3]`（上面加粗数字），和为 17。

示例 3：

输入： `nums = [1,-5,-20,4,-1,3,-6,-3]`, `k = 2`

输出： 0

提示：

- `1 <= nums.length, k <= 105`
- `-104 <= nums[i] <= 104`

题目来源：[力扣 1696. 跳跃游戏 VI](#)。

这题和 [1425. 带限制的子序列和](#) 非常像：

1425 题是让你求最大子序列和，子序列中每两个元素之间的间隔不能超过 `k`；这道题其实也是让你求元素间隔不超过 `k` 的最大子序列和，只不过又多了些限制，即子序列的第一个元素必须是 `nums[0]`，最后一个元素必须是 `nums[-1]`。

你可以想办法复用第 1425 题的解法，不过我这里按照标准的 [动态规划解题框架](#) 来逐步优化，最后通过 [单调队列结构](#) 中给出的单调队列通用实现来优化动态规划解法的效率。

需要注意的是，本题使用标准的动态规划解法也会超时，这并不是动态规划的锅，而是本题给的数据规模太大，需要更先进的数据结构（单调队列）来优化状态转移的效率。

```
// 第一步，暴力递归解法（超时）
class Solution1 {
public:
    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();
        return dp(nums, n - 1, k);
    }

    // 定义：到达 nums[p] 所能获得的最大分数是 dp(nums, p)
    // 能跳到 nums[p]，必然是从 nums[p-k..p-1] 中的某个位置跳来的
    // 故状态转移方程为：dp[p] = max(nums[p-k..p-1]) + nums[p]
    int dp(vector<int>& nums, int p, int k) {
        if (p == 0) {
            return nums[0];
        }
        if (p < 0) {
            return INT_MIN;
        }
        // 实现状态转移方程
        int res = INT_MIN;
        for (int i = 1; i <= k; i++) {
            res = max(res, dp(nums, p - i, k));
        }
        res += nums[p];
        return res;
    }
};

// 第二步，带备忘录的递归解法（超时）
class Solution2 {
public:
    vector<int> memo;

    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();
        memo = vector<int>(n, INT_MIN);
        return dp(nums, n - 1, k);
    }

    // 定义：到达 nums[p] 所能获得的最大分数是 dp(nums, p)
    int dp(vector<int>& nums, int p, int k) {
        if (p == 0) {
            return nums[0];
        }
    }
```

```

    }
    if (p < 0) {
        return INT_MIN;
    }
    // 查备忘录，避免冗余计算
    if (memo[p] != INT_MIN) {
        return memo[p];
    }
    // 实现状态转移方程，结果存入备忘录
    for (int i = 1; i <= k; i++) {
        memo[p] = max(memo[p], dp(nums, p - i, k));
    }
    memo[p] += nums[p];
    return memo[p];
}
};

```

// 第三步，自顶向下的递归改为自底向上的迭代解法（超时）

```

class Solution3 {
public:
    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> dp(n, INT_MIN);
        dp[0] = nums[0];
        for (int p = 1; p < n; p++) {
            for (int i = 1; i <= k; i++) {
                if (p - i < 0) {
                    continue;
                }
                dp[p] = max(dp[p], dp[p - i]);
            }
            dp[p] += nums[p];
        }
        return dp[n - 1];
    }
};

```

// 第四步，利用单调队列结构消除内层循环（通过）

```

class Solution {
public:
    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();
        MonotonicQueue window;
        // 定义：到达 nums[p] 的最大分数为 dp[p]
        vector<int> dp(n, INT_MIN);
        // dp 数组初始化为最小值
        dp[0] = nums[0];
        window.push(dp[0]);
        // 状态转移
        for (int p = 1; p < n; p++) {

```



```

        dp[p] = window.max() + nums[p];
        // 维护窗口装着 dp[p-1..p-k]
        if (p >= k) {
            window.pop();
        }
        window.push(dp[p]);
    }
    return dp[n - 1];
}
};

```

// 单调队列的实现

```

class MonotonicQueue {
    // 常规队列，存储所有元素
    deque<int> q;
    // 元素降序排列的单调队列，头部是最大值
    deque<int> maxq;
    // 元素升序排列的单调队列，头部是最小值
    deque<int> minq;

public:
    // 添加元素到队尾
    void push(int elem) {
        // 维护常规队列，直接在队尾插入元素
        q.push_back(elem);

        // 维护 maxq，将小于 elem 的元素全部删除
        while (!maxq.empty() && maxq.back() < elem) {
            maxq.pop_back();
        }
        maxq.push_back(elem);

        // 维护 minq，将大于 elem 的元素全部删除
        while (!minq.empty() && minq.back() > elem) {
            minq.pop_back();
        }
        minq.push_back(elem);
    }

    // 获取最大值
    int max() {
        // maxq 的头部是最大元素
        return maxq.front();
    }

    // 获取最小值
    int min() {
        // minq 的头部是最小元素
        return minq.front();
    }
}

```

```

// 弹出队首元素
void pop() {
    // 从标准队列头部弹出需要删除的元素
    int deleteVal = q.front();
    q.pop_front();

    // 由于 push 的时候会删除元素，deleteVal 可能已经被删掉了
    if (!maxq.empty() && deleteVal == maxq.front()) {
        maxq.pop_front();
    }
    if (!minq.empty() && deleteVal == minq.front()) {
        minq.pop_front();
    }
}

// 获取队列大小
int size() {
    // 标准队列的大小即是当前队列的大小
    return q.size();
}

// 检查队列是否为空
bool isEmpty() {
    return q.empty();
}
};

```

1425. 带限制的子序列和

给你一个整数数组 `nums` 和一个整数 `k`，请你返回 **非空** 子序列元素和的最大值，子序列需要满足：子序列中每两个 **相邻** 的整数 `nums[i]` 和 `nums[j]`，它们在原数组中的下标 `i` 和 `j` 满足 $i < j$ 且 $j - i \leq k$ 。

数组的子序列定义为：将数组中的若干个数字删除（可以删除 0 个数字），剩下的数字按照原本的顺序排布。

示例 1：

输入： `nums = [10,2,-10,5,20]`, `k = 2`

输出： 37

解释： 子序列为 `[10, 2, 5, 20]`。

示例 2：

输入： `nums = [-1,-2,-3]`, `k = 1`

输出： -1

解释： 子序列必须是非空的，所以我们选择最大的数字。

示例 3:

输入: nums = [10,-2,-10,-5,20], k = 2

输出: 23

解释: 子序列为 [10, -2, -5, 20]。

提示:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

题目来源: [力扣 1425. 带限制的子序列和](#)。

如果你看过前文 [最长递增子序列问题详解](#), 那么这道题的思路应该是很简单的:

设计一个 dp 数组, dp[i] 表示以 nums[i] 为结尾的最大子序列之和, 然后题目要求的答案就是 $\max(\text{dp}[\dots])$ 。

由于题目说两个序列之间的间隔不能超过 k, 所以 dp[i] 的值就可以根据 dp[i-k..i-1] 的值推导出来, 那么状态转移逻辑就是:

```
for (int i = 1; i < n; i++) {
    int maxVal = 0;
    for (int j = 1; j <= k; j++) {
        maxVal = Math.max(maxVal, dp[i - j]);
    }
    dp[i] = maxVal + nums[i];
}
```

想到这一步其实已经可以写出动态规划解法代码了, 但由于测试数据的规模较大, 这个 $O(KN)$ 的算法会超时。

我们注意到内层的 for 循环实际上就是在计算 dp[i-k..i-1] 的最大值, 随着 i 的增加, dp[i-k..i-1] 就像一个滑动前进的窗口, 这个场景不需要傻乎乎用 for 循环遍历求最值, 前文讲到的 [单调队列](#) 可以高效实现这个过程, 将整个算法的复杂度降低到 $O(N)$ 。

我将给出经过单调队列优化的解法代码和未经优化的解法代码, 注意单调队列在其中起到的作用。

```
#include <vector>
#include <deque>
#include <climits>
using namespace std;

// 下面给出的是单调队列的通用实现, 运行速度可能较慢, 你可以自行简化提升速度
// 单调队列的详细解析见
// https://labuladong.online/algo/problem-set/monotonic-queue/
template<typename E>
```

```

class MonotonicQueue {
    // 常规队列，存储所有元素
    deque<E> q;
    // 元素降序排列的单调队列，头部是最大值
    deque<E> maxq;
    // 元素升序排列的单调队列，头部是最小值
    deque<E> minq;

public:
    void push(E elem) {
        // 维护常规队列，直接在队尾插入元素
        q.push_back(elem);

        // 维护 maxq，将小于 elem 的元素全部删除
        while (!maxq.empty() && maxq.back() < elem) {
            maxq.pop_back();
        }
        maxq.push_back(elem);

        // 维护 minq，将大于 elem 的元素全部删除
        while (!minq.empty() && minq.back() > elem) {
            minq.pop_back();
        }
        minq.push_back(elem);
    }

    E max() {
        // maxq 的头部是最大元素
        return maxq.front();
    }

    E min() {
        // minq 的头部是最大元素
        return minq.front();
    }

    E pop() {
        // 从标准队列头部弹出需要删除的元素
        E deleteVal = q.front();
        q.pop_front();

        // 由于 push 的时候会删除元素，deleteVal 可能已经被删掉了
        if (deleteVal == maxq.front()) {
            maxq.pop_front();
        }
        if (deleteVal == minq.front()) {
            minq.pop_front();
        }
        return deleteVal;
    }
}

```

```

int size() {
    // 标准队列的大小即是当前队列的大小
    return q.size();
}

bool isEmpty() {
    return q.empty();
}
};

// 经过单调队列优化的动态规划解法
class Solution {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        int n = nums.size();
        // 定义: dp[i] 表示以 nums[i] 结尾的子序列的最大和
        vector<int> dp(n);
        dp[0] = nums[0];
        // 单调队列辅助计算 dp[i-k..i-1] 的最大值
        MonotonicQueue<int> window;
        window.push(dp[0]);

        for (int i = 1; i < n; i++) {
            // 状态转移方程
            dp[i] = max(nums[i], window.max() + nums[i]);
            // 维护滑动窗口的大小为 k
            if (window.size() == k) {
                window.pop();
            }
            window.push(dp[i]);
        }
        // dp 数组中的最大值就是结果
        int res = INT_MIN;
        for (int i = 0; i < n; i++) {
            res = max(res, dp[i]);
        }
        return res;
    }
};

```

// 未经优化的动态规划解法，超时

```

class Solution2 {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        int n = nums.size();
        // 定义: dp[i] 表示以 nums[i] 结尾的子序列的最大和
        vector<int> dp(n);
        // base case, 以 nums[0] 结尾的子序列只有它本身
        dp[0] = nums[0];
    }
};

```

```

// 状态转移方程
for (int i = 1; i < n; i++) {
    int maxVal = 0;
    for (int j = 1; j <= k; j++) {
        if (i - j < 0) {
            continue;
        }
        maxVal = max(maxVal, dp[i - j]);
    }
    // dp[i] 的值可以根据 dp[i-k..i-1] 的最大值推导出来
    dp[i] = maxVal + nums[i];
}

// dp 数组中的最大值就是结果
int res = INT_MIN;
for (int i = 0; i < n; i++) {
    res = max(res, dp[i]);
}
return res;
}
};

```